

Lập Lịch CPU



NỘI DUNG

01

Tổng quan về lập lịch CPU

Khái niệm và thuật ngữ

02

Các tiêu chí lập lịch

Cơ sở xây dựng qui trình điều phối CPU

03

Các giải thuật lập lịch

Đặc điểm của từng giải thuật

04

Luyện tập

Vận dụng thực hành tổng hợp

05

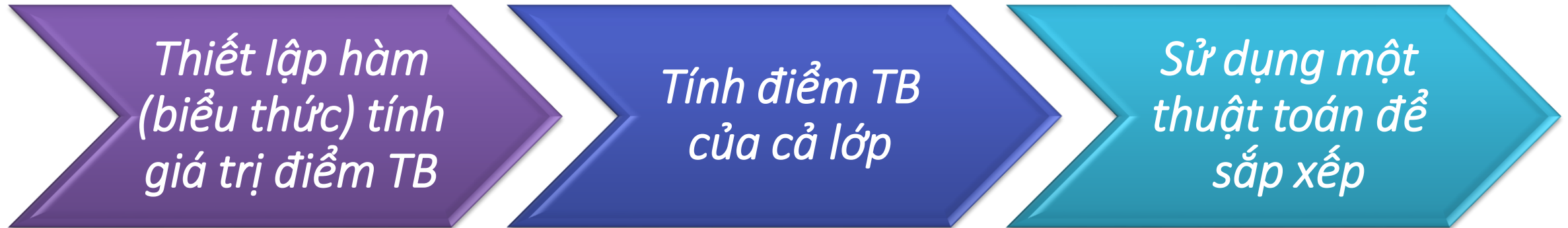
Tổng kết

Tổng kết các nội dung chính cần lưu ý



❖ Phân biệt chương trình và tiến trình:

Xét bài toán thực hiện chương trình xếp thứ hạng học sinh trong lớp theo điểm TB các môn trong năm học đó



Tổng Quan Về Lập Lịch CPU



❖ Bộ nhớ của tiến trình:

Được chia làm 4 phần

- Dành riêng cho biến cục bộ

Ngăn xếp

Vùng nhớ linh hoạt

- Phân bổ bộ nhớ động
- Quản lý và gọi bởi lệnh new, delete,...

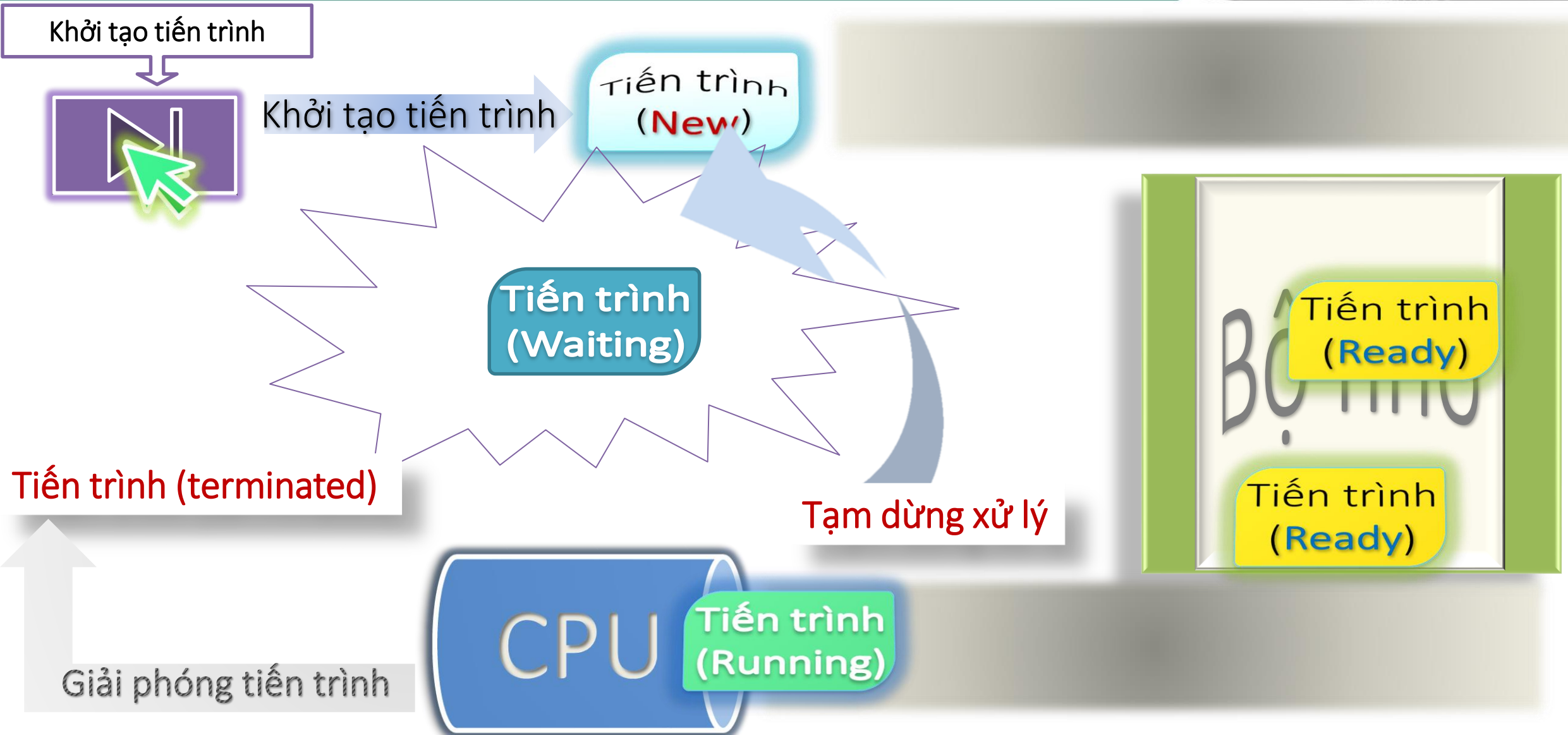
- Phân phối trước khi chương trình thực thi
- Biến toàn cục, biến tĩnh,...

Lớp dữ liệu

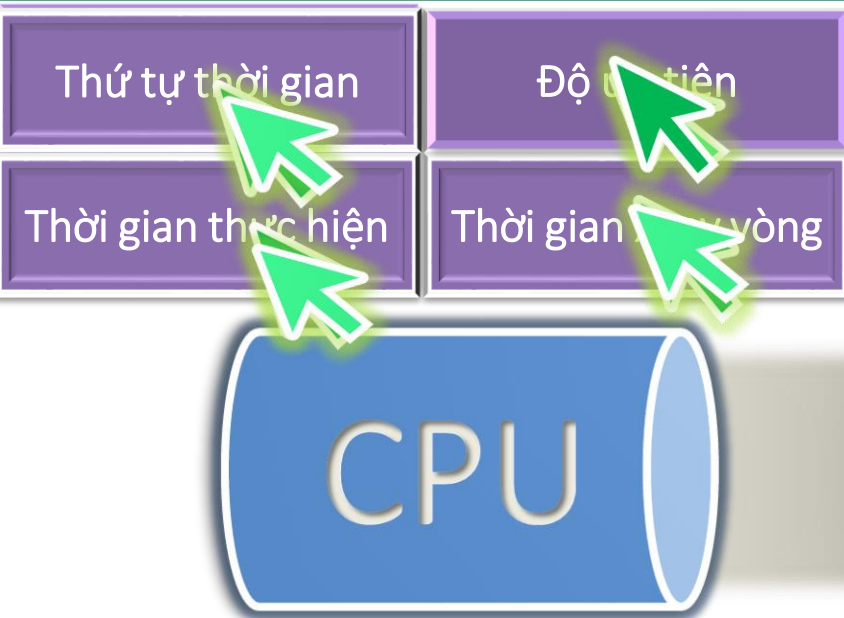
Tập văn bản

- Lưu trữ cố định
- Mã chương trình,...

Tổng Quan Về Lập Lịch CPU



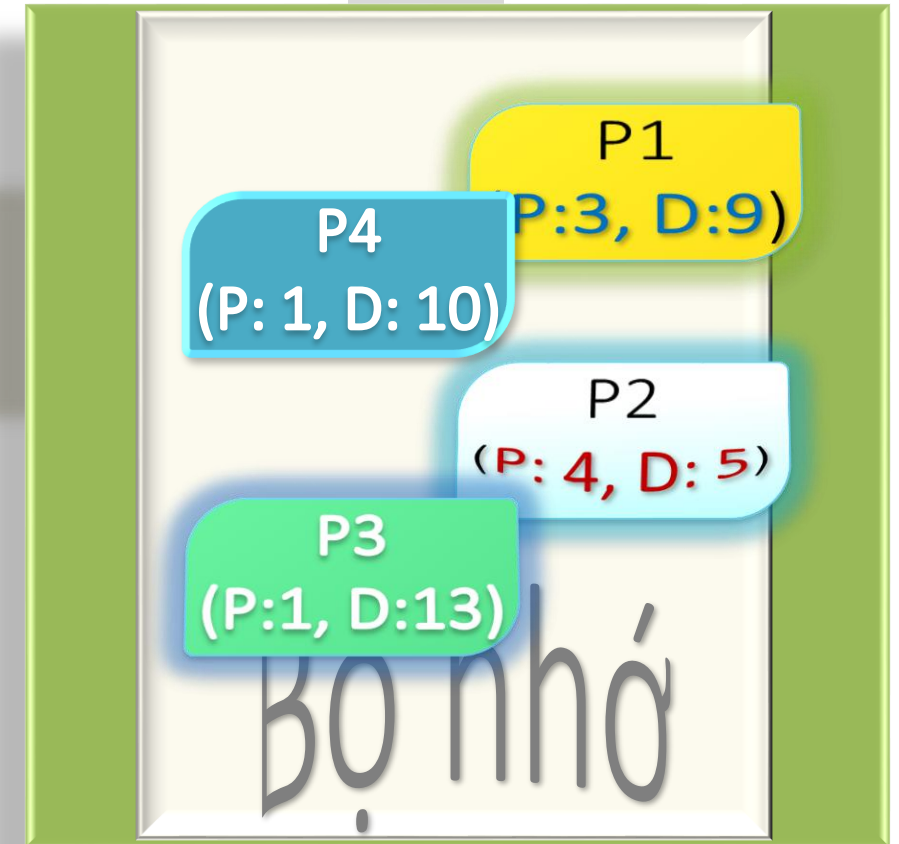
Tổng Quan Về Lập Lịch CPU



P_i: Thứ tự các tiến trình được xếp vào bộ nhớ

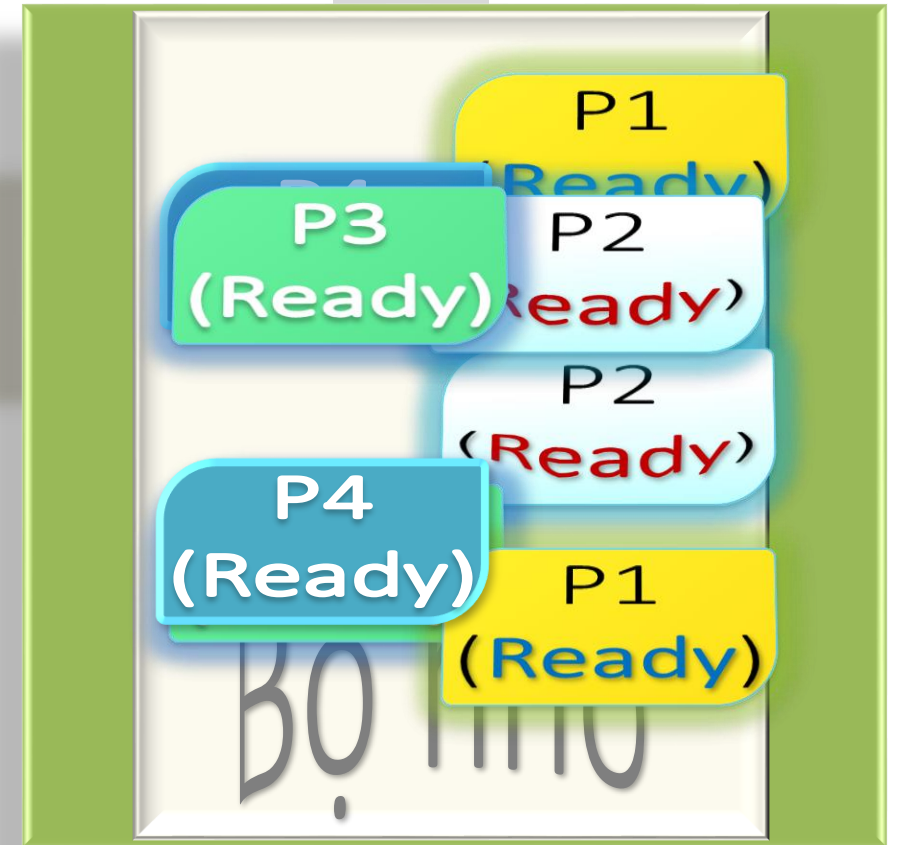
P: Priority (Độ ưu tiên)

D: Duration (Thời gian xử lý)



Tổng Quan Về Lập Lịch CPU

❖ *Xử lý xoay vòng:*



Tổng Quan Về Lập Lịch CPU



❖ Lập lịch CPU:

- Lập lịch trình các quy trình/công việc được thực hiện để hoàn thành công việc đúng thời hạn.
- **Lập lịch CPU là một quá trình cho phép một tiến trình sử dụng CPU trong khi một tiến trình khác bị trì hoãn** (ở chế độ chờ) do không có sẵn bất kỳ tài nguyên nào như I / O, v.v., do đó tận dụng tối đa CPU.
- Mục đích của Lập lịch CPU là làm cho hệ thống hiệu quả hơn, nhanh hơn và công bằng hơn.
- Bất cứ khi nào CPU không hoạt động, hệ điều hành phải chọn một trong các tiến trình trong dòng sẵn sàng để khởi chạy.
- Quá trình lựa chọn được thực hiện bởi bộ lập lịch (CPU) tạm thời.

Tổng Quan Về Lập Lịch CPU

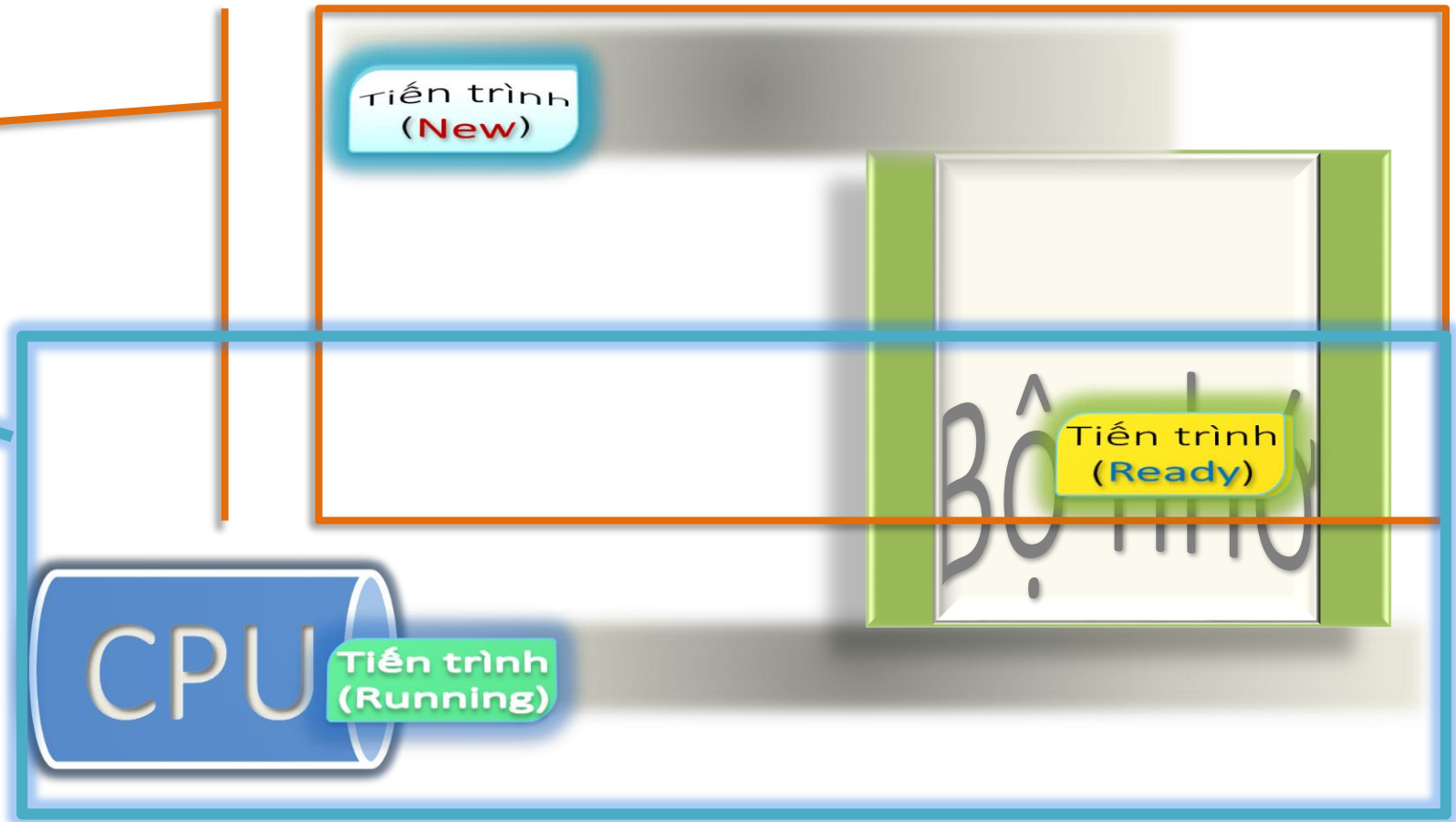


❖ Lập lịch CPU:

- Lập lịch tiến trình là quá trình quản lý quy trình xử lý việc loại bỏ một tiến trình đang hoạt động khỏi CPU và chọn một tiến trình khác dựa trên một chiến lược cụ thể.

- Phân loại bộ lập lịch:

- Dài hạn
- Trung gian
- Ngắn hạn



Tổng Quan Về Lập Lịch CPU

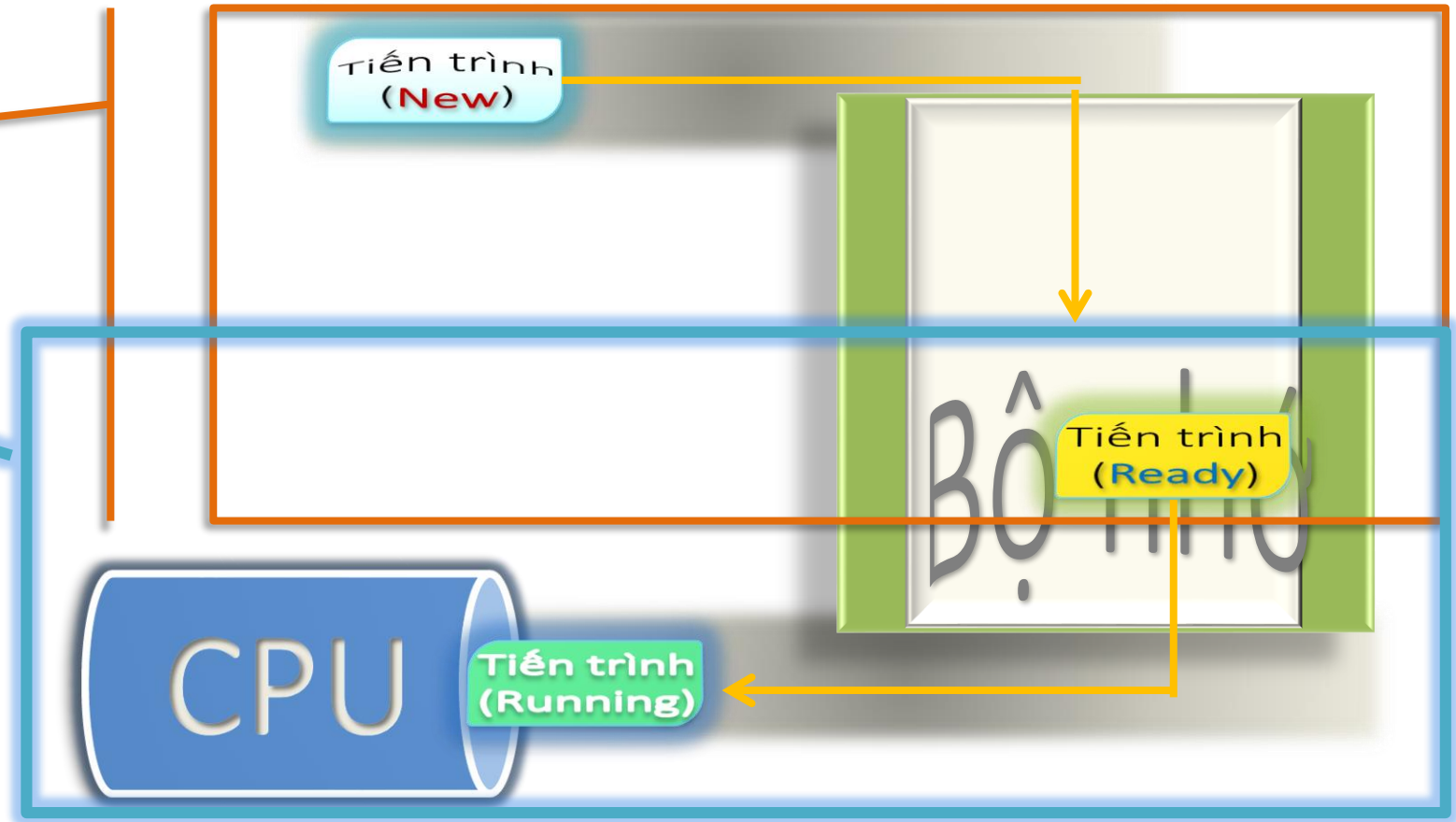


❖ Lập lịch CPU:

➤ Lập lịch tiến trình là quá trình quản lý quy trình xử lý việc loại bỏ một tiến trình đang hoạt động khỏi CPU và chọn một tiến trình khác dựa trên một chiến lược cụ thể.

➤ Phân loại bộ lập lịch:

- Dài hạn
- Trung gian
- Ngắn hạn





❖ Các thuật ngữ sử dụng:

- **Thời gian đến:** Thời điểm tiến trình đến hàng đợi sẵn sàng.
- **Thời gian hoàn thành:** Thời điểm mà tiến trình hoàn thành quá trình thực thi của nó.
- **Thời gian bùng nổ:** Thời gian được yêu cầu bởi một quá trình để thực thi CPU.
- **Thời gian quay vòng:** Thời gian Chênh lệch giữa thời gian hoàn thành và thời gian đến.

$$\text{Thời gian quay vòng} = \text{Thời gian hoàn thành} - \text{Thời gian đến}$$

- **Thời gian chờ (W.T):** Thời gian Chênh lệch giữa thời gian quay vòng và thời gian bùng nổ.

$$\text{Thời gian chờ đợi} = \text{Thời gian quay vòng} - \text{Thời gian bùng nổ}$$





❖ Đồng hồ ngắt thời gian:

- ✓ Bộ đếm thời gian quy định một thông số thời gian t thích hợp ứng với một lượt cấp CPU cho một tiến trình
- ✓ Sau một khoảng thời gian t sẽ xảy ra một ngắt báo hiệu hết thời gian sử dụng CPU của tiến trình hiện hành. HĐH sẽ thu hồi CPU và bộ điều phối sẽ quyết định tiến trình nào sẽ được cấp phát



❖ Độ ưu tiên của tiến trình:

- ✓ Là giá trị giúp phân định tầm quan trọng của tiến trình
 - Độ ưu tiên tĩnh :
 - Được gán sẵn cho tiến trình khi mới được tạo ra
 - Không thay đổi
 - Độ ưu tiên động : thay đổi theo thời gian và môi trường xử lý của tiến trình



❖ Mục tiêu của các thuật toán điều phối tiến trình:

Các tiến trình cần được điều phối hợp lý để giúp cho hệ thống thực hiện hiệu quả hơn ở các vấn đề sau:

- ✓ Sự công bằng giữa các tiến trình
- ✓ Tính hiệu quả
- ✓ Cực tiểu hóa thời gian lưu lại trong hệ thống
- ✓ Thời gian đáp ứng hợp lý
- ✓ Thông lượng tối đa

Tiêu Chí Điều Phối Tiến Trình



❖ Lập kế hoạch điều phối CPU:

- Lập kế hoạch ưu tiên:
 - ✓ Được sử dụng khi một tiến trình chuyển từ trạng thái **chạy** sang trạng thái **sẵn sàng** hoặc từ trạng thái **chờ** sang trạng thái **sẵn sàng**.
- Lập lịch không ưu tiên:
 - ✓ Được sử dụng khi một tiến trình **kết thúc** hoặc khi một quá trình chuyển từ trạng thái **chạy** sang trạng thái **chờ**.

Tiêu Chí Điều Phối Tiến Trình



- **Sử dụng CPU:** Mục đích chính của bất kỳ thuật toán CPU nào là giữ cho CPU bận rộn nhất có thể.
- **Thông lượng:** Hiệu suất CPU trung bình là số lượng tiến trình được thực hiện và hoàn thành trong mỗi đơn vị.
- **Thời gian quay vòng:** Thời gian trôi qua kể từ thời điểm chuyển giao tiến trình đến thời điểm hoàn thành được gọi là thời gian quay vòng. Đó lượng thời gian chờ truy cập bộ nhớ, xếp hàng chờ, sử dụng CPU và chờ I/O.
- **Thời gian chờ:** Thời gian dành cho quá trình chờ đợi trong hàng đợi sẵn sàng.
- **Thời gian phản hồi:** Thời gian thực hiện tiến trình khởi tạo cho đến khi nhận được phản hồi đầu tiên. Biện pháp này được gọi là thời gian đáp ứng.

Tiêu Chí Điều Phối Tiến Trình



- ❖ **Thời gian:** Căn cứ theo
 - ✓ Thứ tự tiến trình vào hàng đợi
 - ✓ Thời gian hoàn thành tiến trình
- ❖ **Yêu cầu của tiến trình:** Căn cứ được lựa chọn dựa vào
 - ✓ Thứ tự mức độ ưu tiên của tiến trình
- ❖ **Sự công bằng:** Cần thực hiện
 - ✓ Đáp ứng mọi tiến trình
 - ✓ Theo thời gian chờ



CÁC GIẢI THUẬT LẬP LỊCH

01 FCFS-First come First served

02 Công việc ngắn nhất trước – Shortest
Job First (SJF)

03 Độ ưu tiên (Priority Scheduling)

04 Luân phiên (Round Robin Scheduling)

05 Hàng đợi đa cấp (Multilevel Queue
Scheduling)

06 Hàng đợi phản hồi đa cấp (Multilevel
Feedback Queue)



**FCFS-First come
First served**



FCFS: First-Come, First-Served



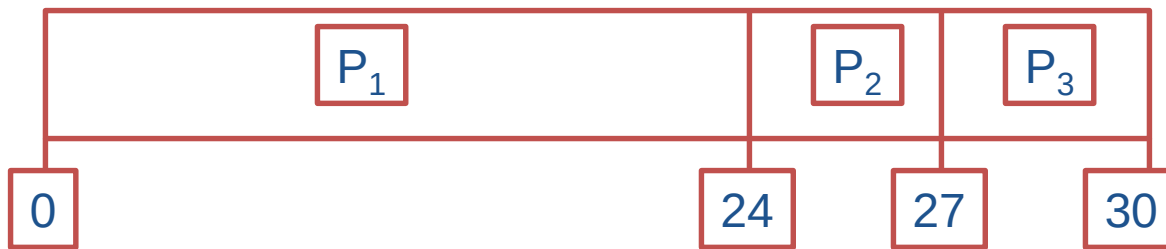
- Tiến trình nào đến trước thì được thực thi trước

- Ví dụ:

	<u>Tiến trình</u>	<u>Thời gian bùng nổ</u>
	P_1	24
	P_2	3
	P_3	3

- Nếu các tiến trình đến theo thứ tự: P_1 , P_2 , P_3

- Biểu đồ Gantt cho quá trình lập lịch:



- Thời gian chờ: $P_1 = 0$; $P_2 = 24$; $P_3 = 27$

- Thời gian chờ trung bình: $(0 + 24 + 27)/3 = 17$

t	P1	P2	P3
0	24	3	3
24	0		
27		0	
30			0

FCFS: First-Come, First-Served

Minh họa lần lượt

- Tiến trình nào đến trước thì được thực thi trước

- Ví dụ:

Tiến trình Thời gian bùng nổ

P_1 24

P_2 3

P_3 3

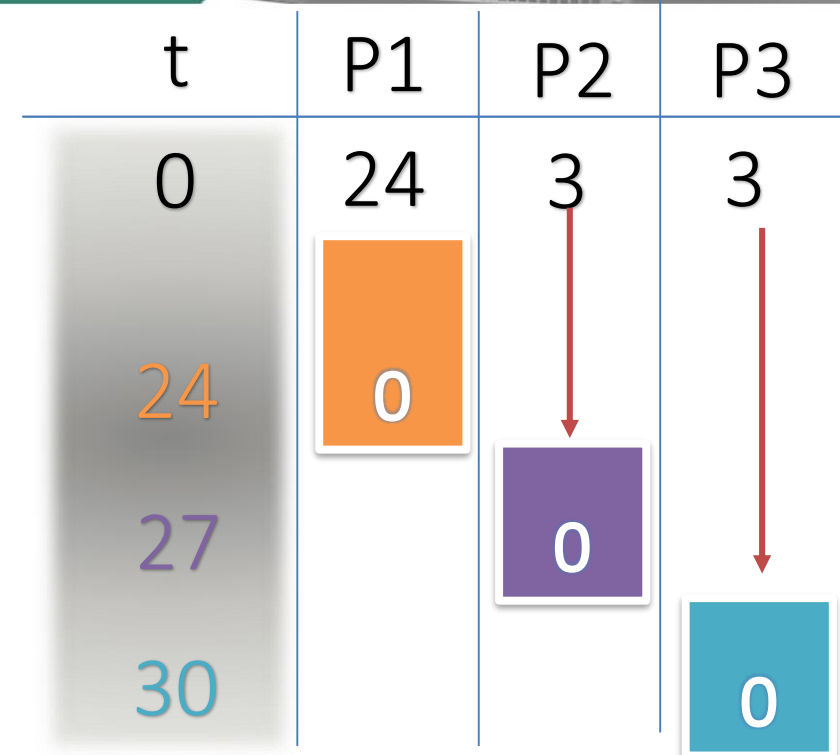
- Nếu các tiến trình đến theo thứ tự: P_1 , P_2 , P_3

- Biểu đồ Gantt cho quá trình lập lịch:



- Thời gian chờ: $P_1 = 0$; $P_2 = 24$; $P_3 = 27$

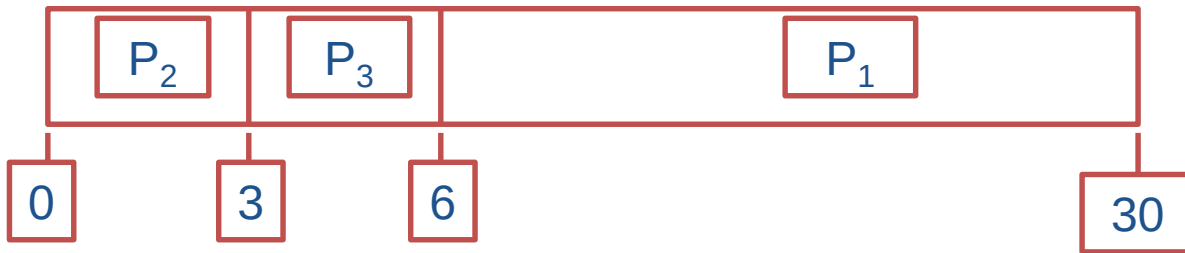
- Thời gian chờ trung bình: $(0 + 24 + 27)/3 = 17$



FCFS: First-Come, First-Served



- Nếu các tiến trình đến theo thứ tự:
 P_2, P_3, P_1
- Biểu đồ Gantt cho quá trình lập lịch là:



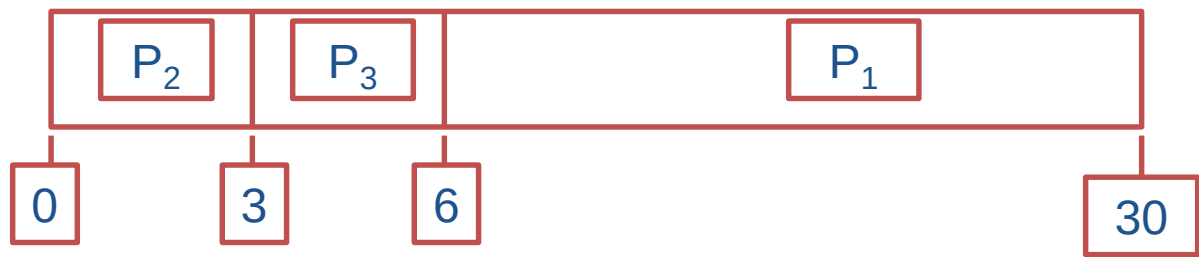
- Thời gian chờ: $P_1 = 6$; $P_2 = 0$; $P_3 = 3$
- Thời gian chờ trung bình: $(6 + 0 + 3)/3 = 3$
- Tốt hơn so với trường hợp trước.
- Hạn chế: Tất cả các tiến trình ngắn chờ 1 tiến trình dài

t	P1	P2	P3
0	24	3	3
3		0	
6			0
30	0		

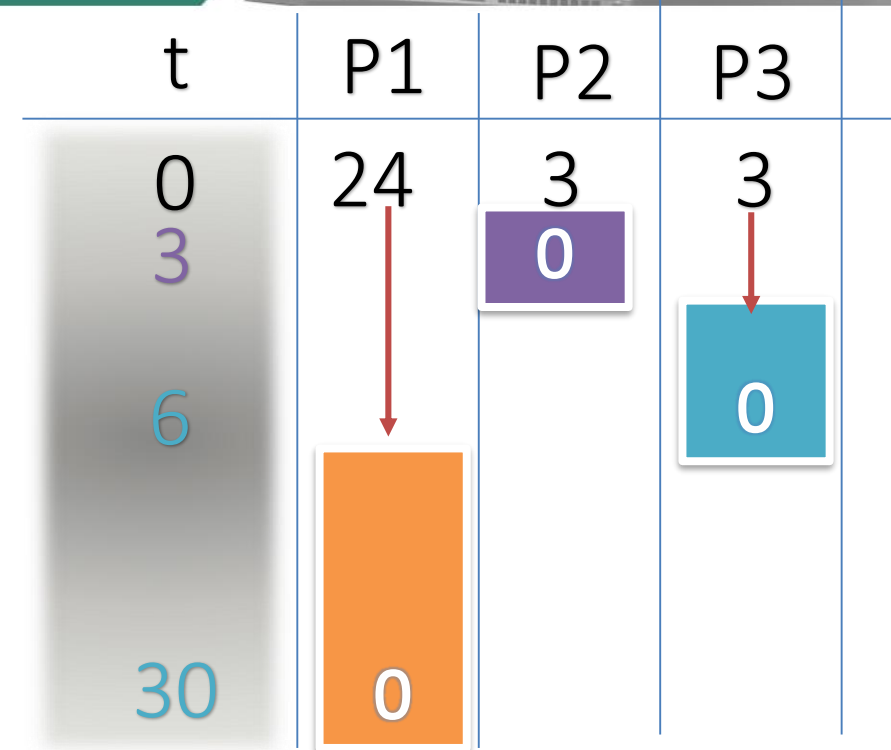
FCFS: First-Come, First-Served

Minh họa lần lượt

- Nếu các tiến trình đến theo thứ tự: P_2, P_3, P_1
- Biểu đồ Gantt cho quá trình lập lịch là:



- Thời gian chờ: $P_1 = 6; P_2 = 0; P_3 = 3$
- Thời gian chờ trung bình: $(6 + 0 + 3)/3 = 3$
- Tốt hơn so với trường hợp trước.
- Hạn chế: Tất cả các tiến trình ngắn chờ 1 tiến trình dài



**Công việc ngắn
nhất trước –
Shortest Job First
(SJF)**



SJF: Shortest-Job-First



- Khi CPU được tự do, nó sẽ được cấp phát cho tiến trình yêu cầu ít thời gian nhất để kết thúc
- Hai giải pháp:
 - **Độc quyền** – Một CPU đã cung cấp cho một tiến trình nào thì phải chờ cho tiến trình đó xử lý xong mới lấy lại.
 - **Không độc quyền** – Một tiến trình mới đến nhưng có thời gian yêu cầu ngắn hơn thời gian còn lại của tiến trình đang chạy, khi đó phải dừng hoạt động để chuyển cho tiến trình mới. Giống mô hình Shortest-Remaining-Time-First (SRTF).
- **SJF** là giải thuật tối ưu – cho phép đạt được thời gian chờ trung bình cực tiểu.

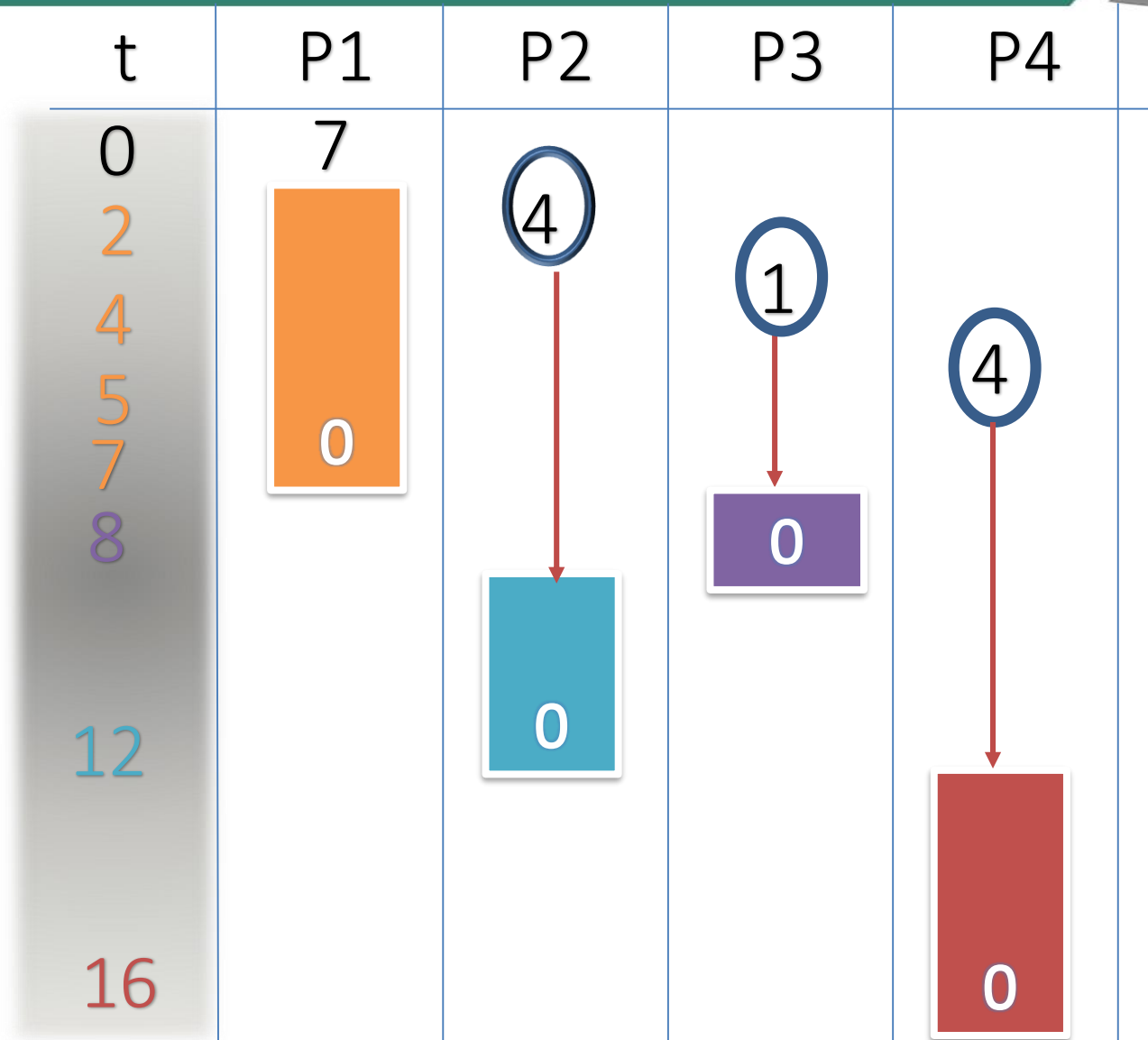
SJF: Shortest-Job-First



<u>Tiến trình</u>	<u>Thời điểm đến</u>	<u>Thời gian bùng nổ</u>
P_1	0.0	7
P_2	2.0	4
P_3	4.0	1
P_4	5.0	4

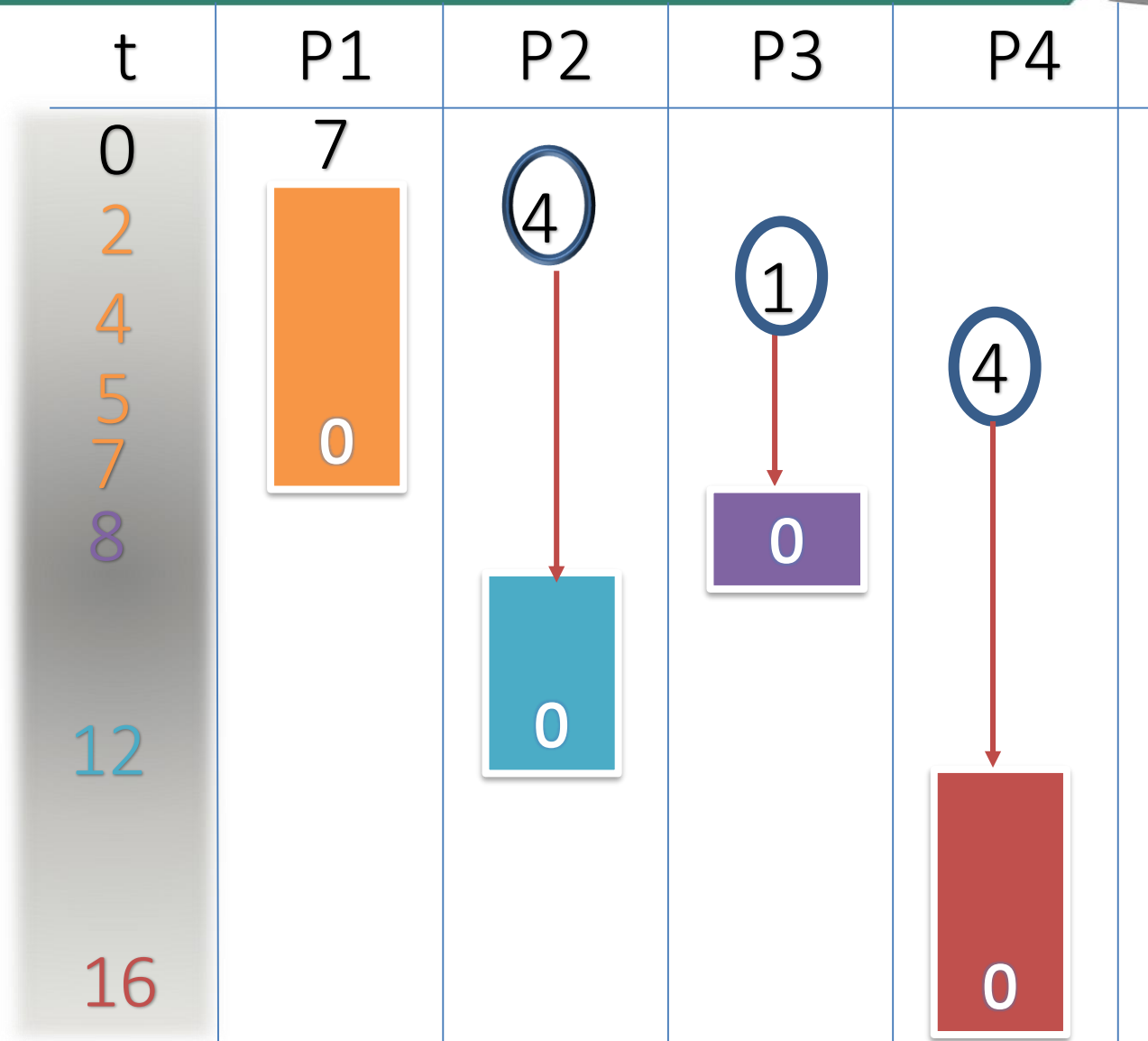
SJF: Shortest-Job-First

Ví dụ 1: trường hợp
độc quyền



SJF: Shortest-Job-First

Ví dụ 1: trường hợp
độc quyền



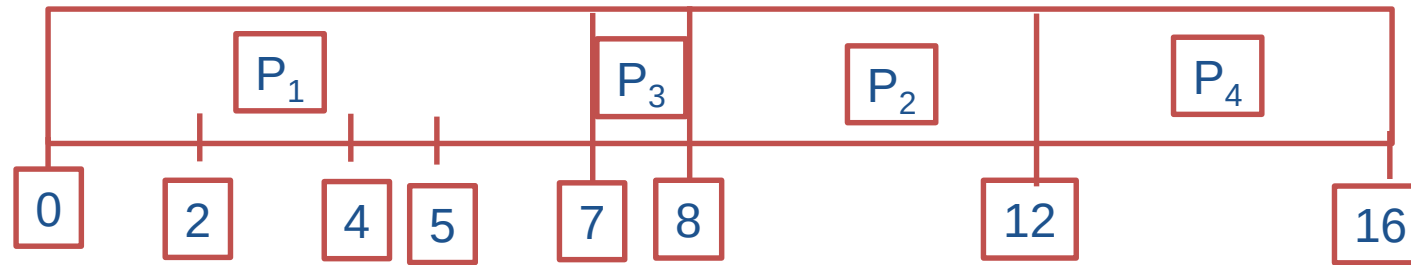
Minh họa lần lượt

SJF: Shortest-Job-First



Ví dụ 1: trường hợp
độc quyền

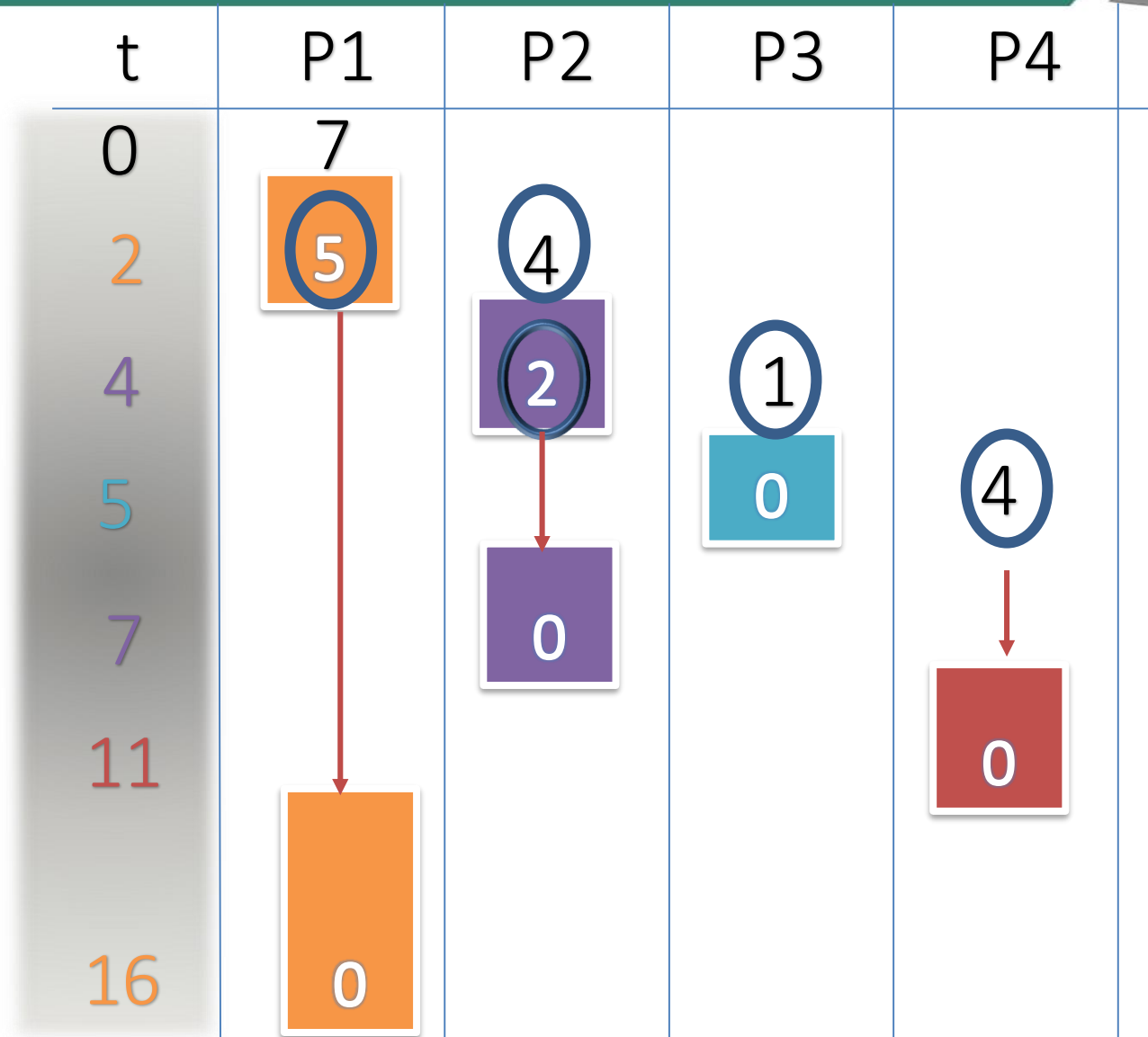
Biểu đồ Gantt:



Thời gian chờ trung bình = $(0 + 6 + 3 + 7)/4 = 4$

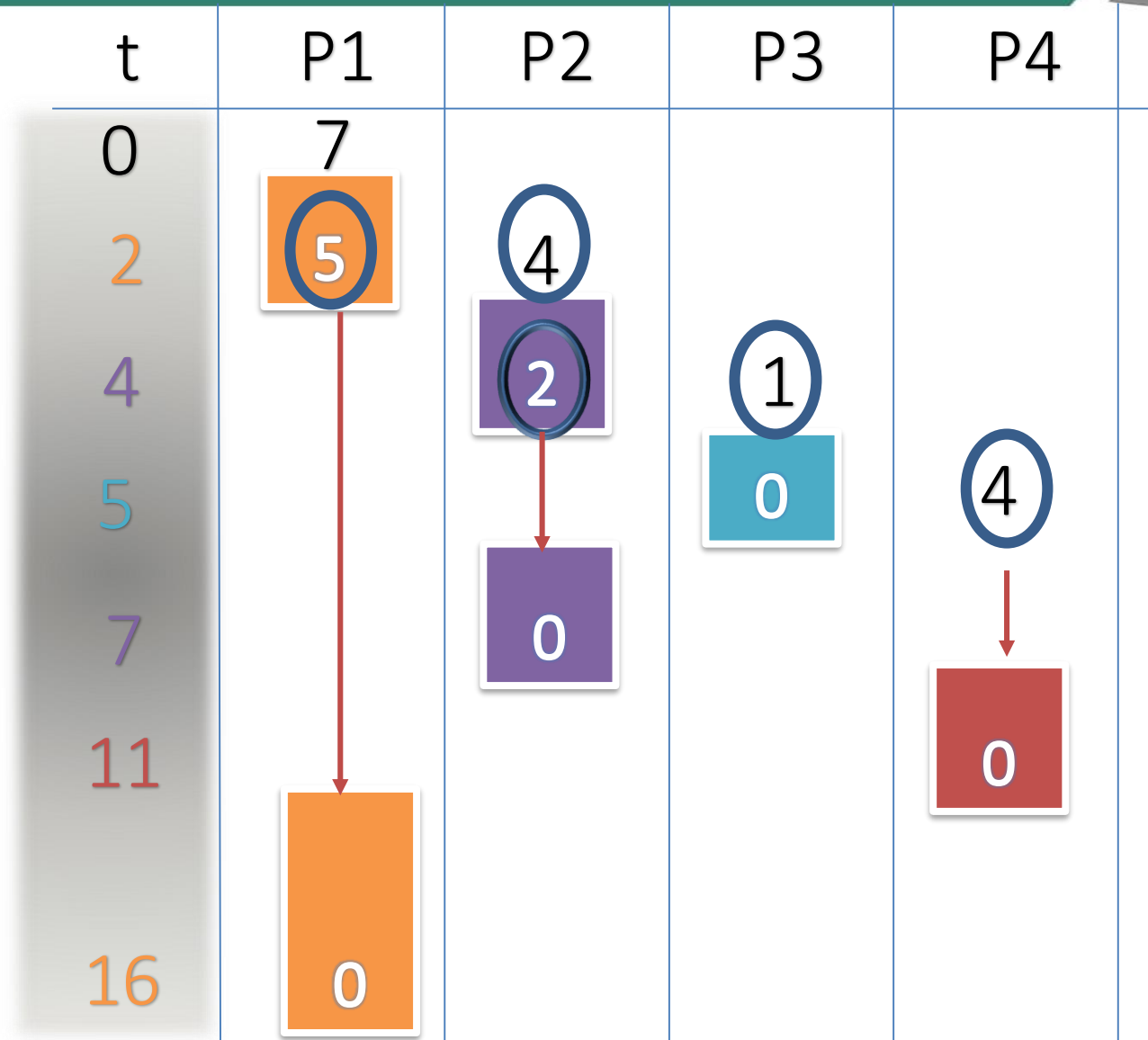
SJF: Shortest-Job-First

Ví dụ 1: trường hợp
không độc quyền



SJF: Shortest-Job-First

Ví dụ 1: trường hợp
không độc quyền



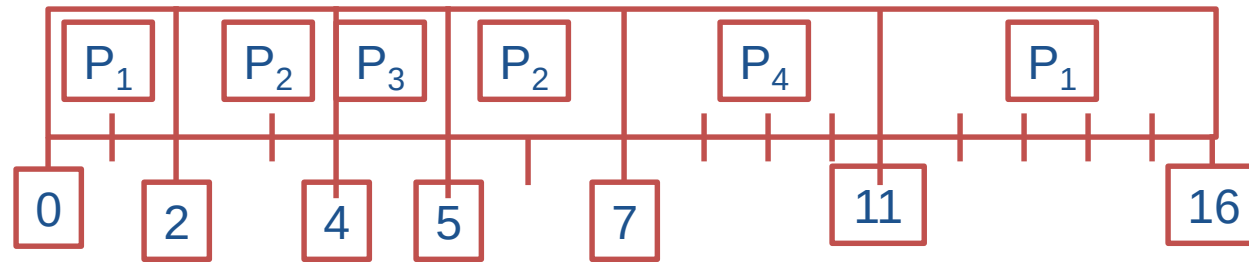
Minh họa lần lượt

SJF: Shortest-Job-First



Ví dụ 1: trường hợp
không độc quyền

Biểu đồ Gantt:



Thời gian chờ trung bình = $(9 + 1 + 0 + 2)/4 = 3$

SJF: Shortest-Job-First



Ví dụ 2a: trường hợp
không độc quyền

Tiến trình

P_1

P_2

P_3

P_4

Thời điểm đến

0.0

3.0

4.0

6.0

Thời gian bùng nổ

4

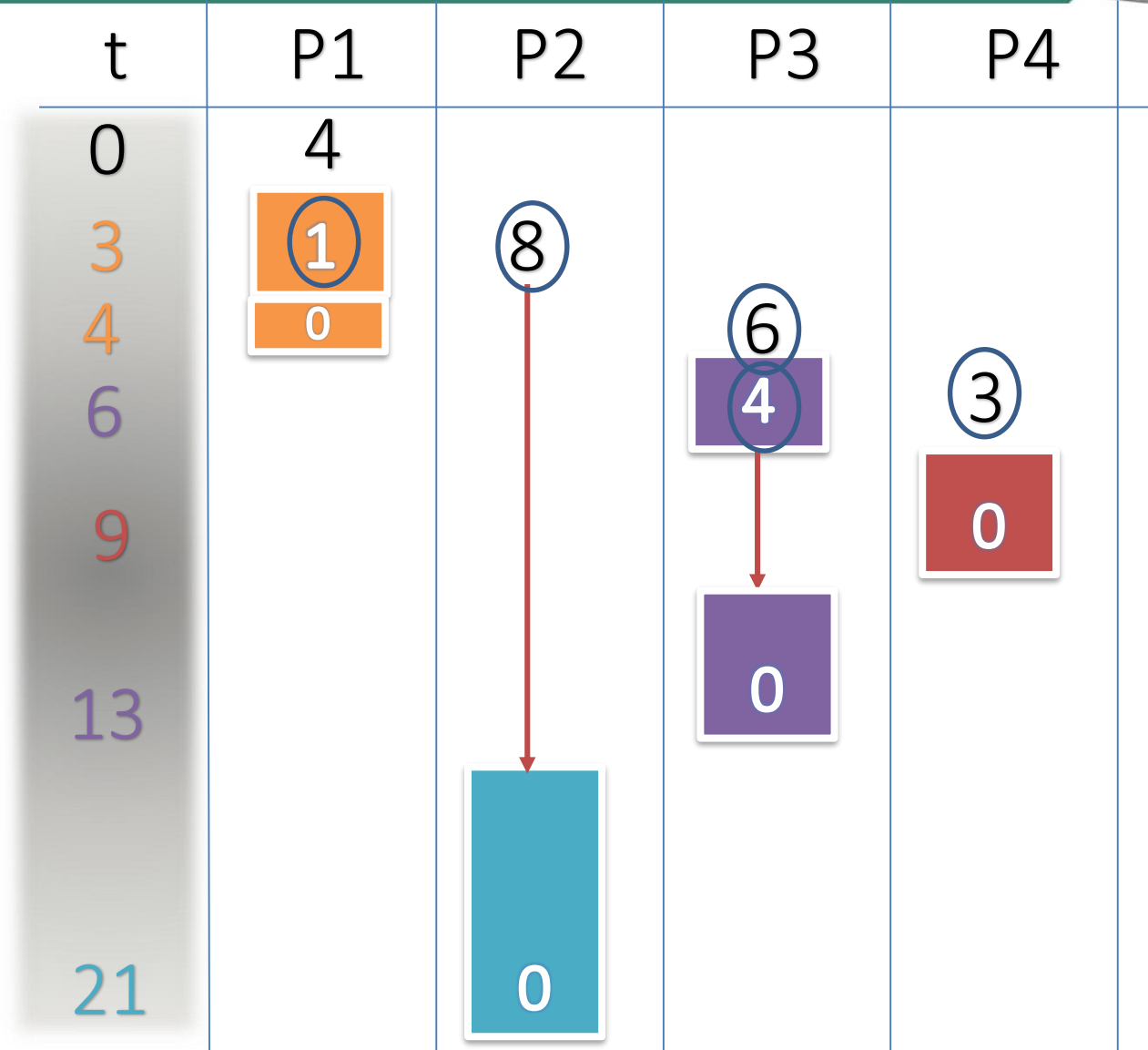
8

6

3

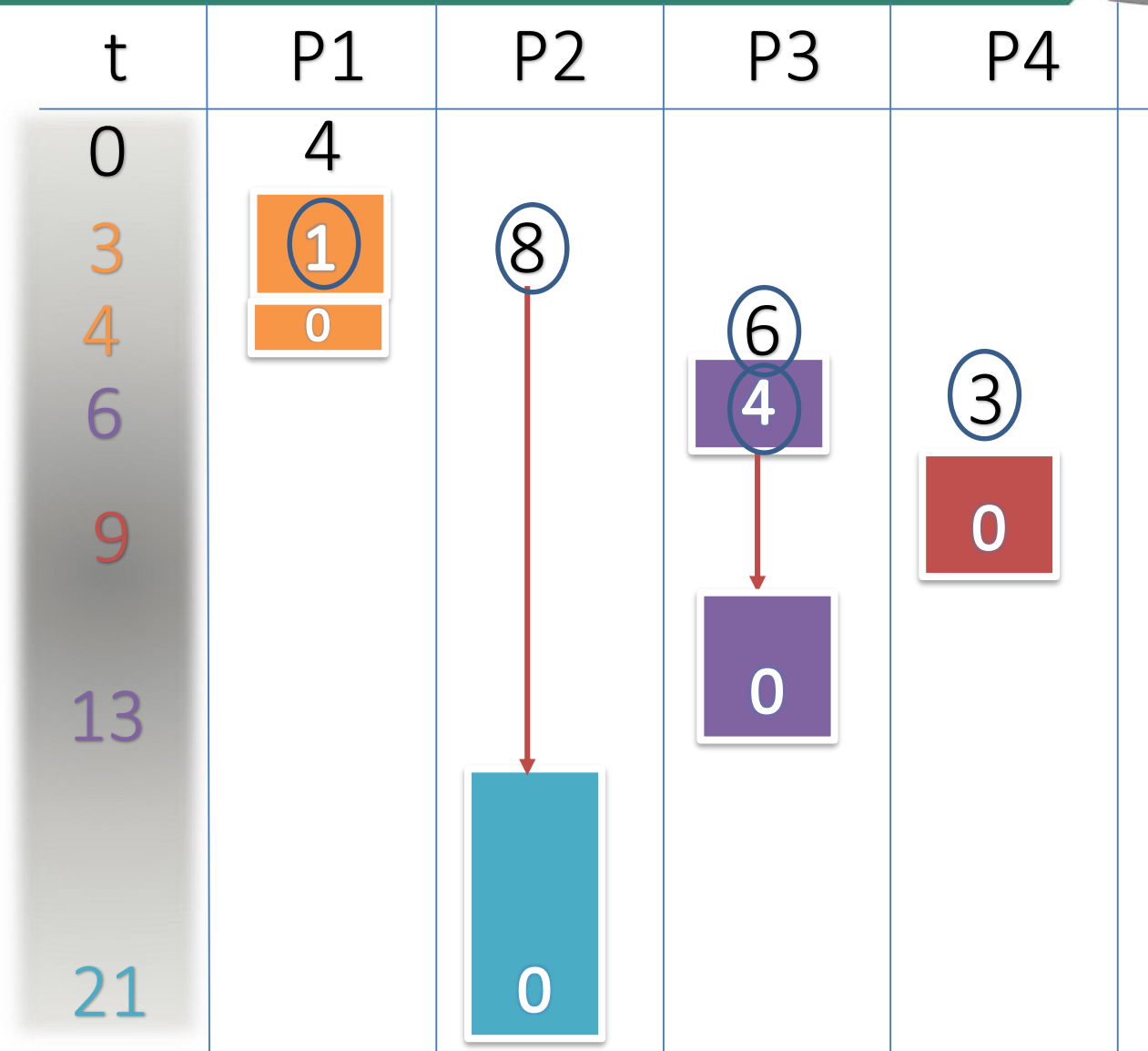
SJF: Shortest-Job-First

Ví dụ 2a: trường hợp
không độc quyền



SJF: Shortest-Job-First

Ví dụ 2a: trường hợp không độc quyền



Minh họa lần lượt

SJF: Shortest-Job-First



Ví dụ 2b: trường hợp
không độc quyền

Tiến trình

P_1

P_2

P_3

P_4

Thời điểm đến

0.0

3.0

4.0

6.0

Thời gian bùng nổ

8

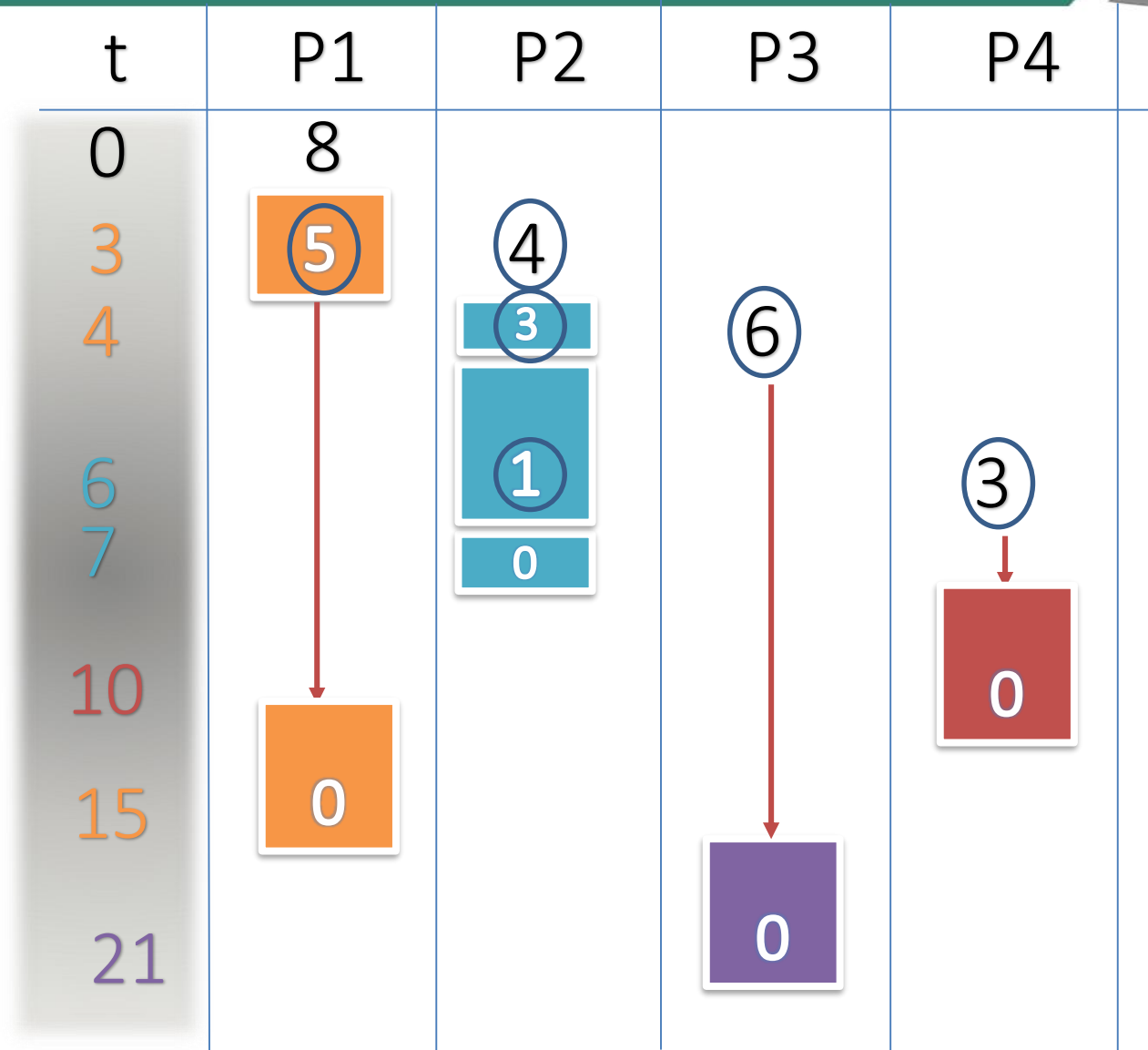
4

6

3

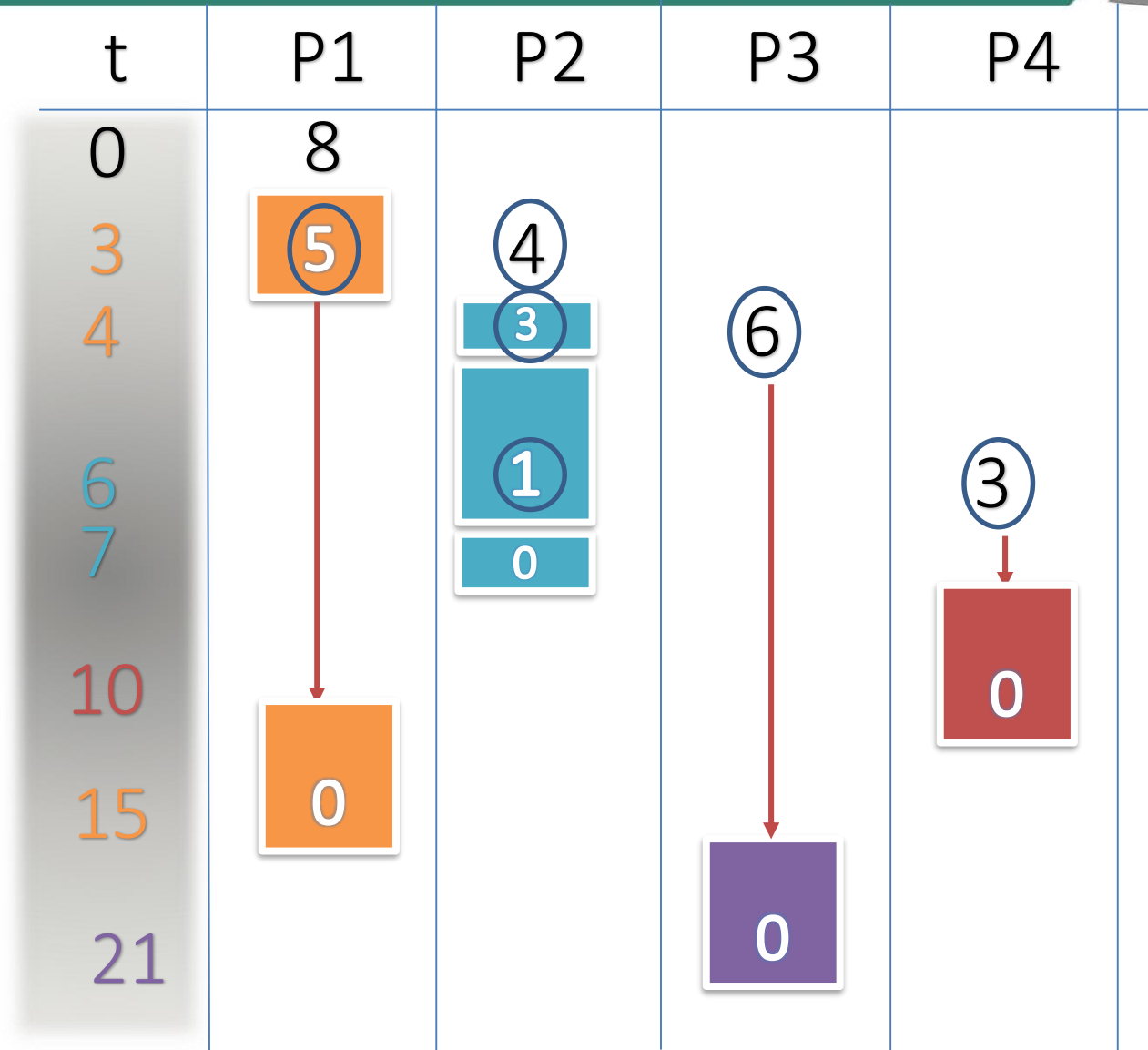
SJF: Shortest-Job-First

Ví dụ 2b: trường hợp
không độc quyền



SJF: Shortest-Job-First

Ví dụ 2b: trường hợp
không độc quyền



Minh họa lần lượt

SJF: Shortest-Job-First



Ví dụ 3: trường hợp
không độc quyền

<u>Tiến trình</u>	<u>Thời điểm đến</u>	<u>Thời gian bùng nổ</u>
P_1	0.0	4
P_2	5.0	8
P_3	6.0	6
P_4	8.0	3

Độ ưu tiên (Priority Scheduling)



Priority Scheduling



- Mỗi tiến trình có 1 độ ưu tiên
- CPU sẽ được điều phối cho tiến trình có độ ưu tiên cao nhất
 - Độc quyền
 - Không độc quyền
- SJF cũng là một dạng của bộ lập lịch theo mức độ ưu tiên, cụ thể mức độ ưu tiên là thời gian cần để hoàn thành tiến trình.
- Vấn đề phát sinh: Tình trạng “**đói CPU**” – Các tiến trình có độ ưu tiên thấp có thể phải chờ đợi vô thời hạn.
- Cách giải quyết $\equiv$ Bộ lập lịch sẽ giảm dần độ ưu tiên của tiến trình này sau mỗi ngắt đồng hồ.

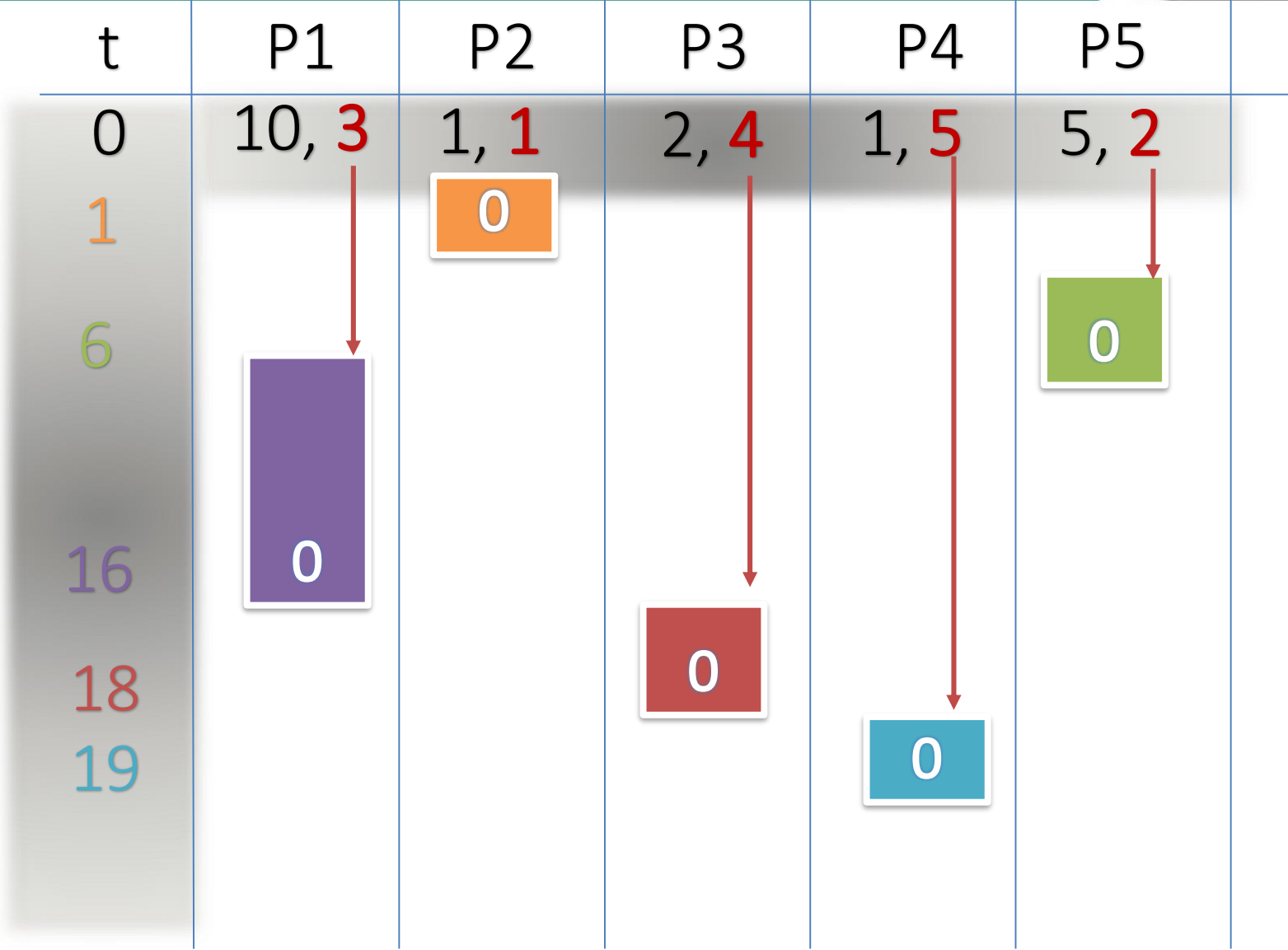
Priority Scheduling



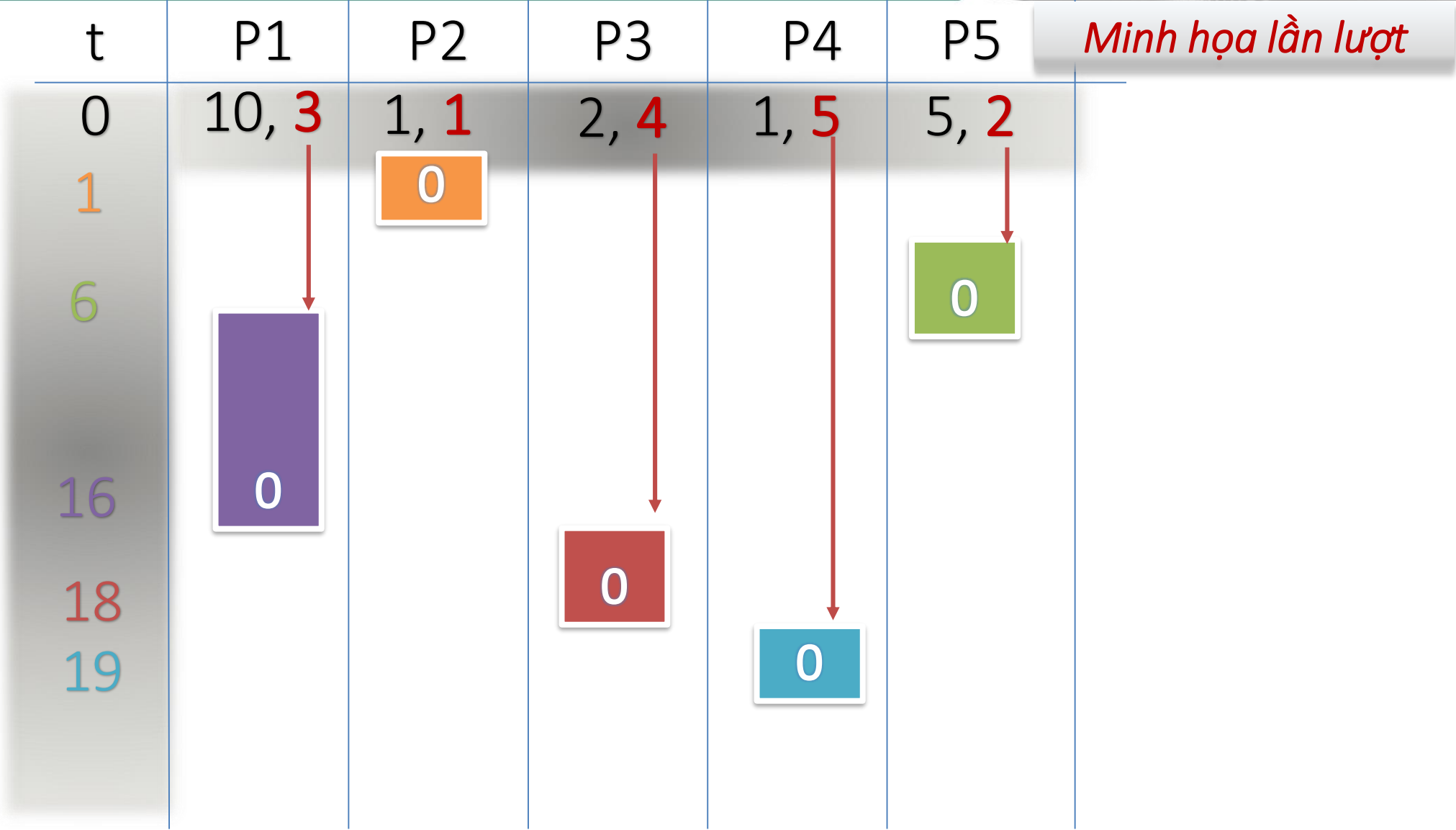
➤ Ví dụ:

<u>Tiến trình</u>	<u>Thời gian bùng nổ</u>	<u>Độ ưu tiên</u>
P_1	10.0	3
P_2	1.0	1
P_3	2.0	4
P_4	1.0	5
P_5	5.0	2

Priority Scheduling



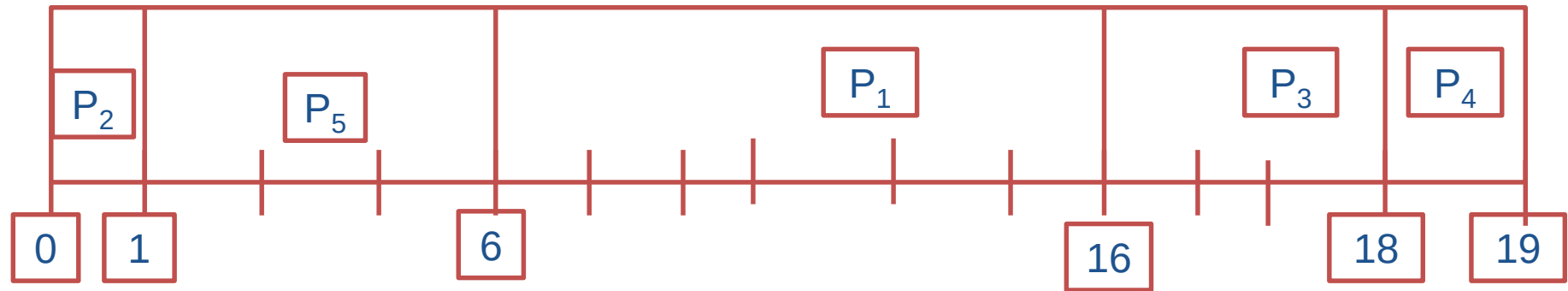
Priority Scheduling



Priority Scheduling



Biểu đồ Gantt



➤ Thời gian chờ trung bình: $(6+0+16+18+1)/5=8.2$

Priority Scheduling



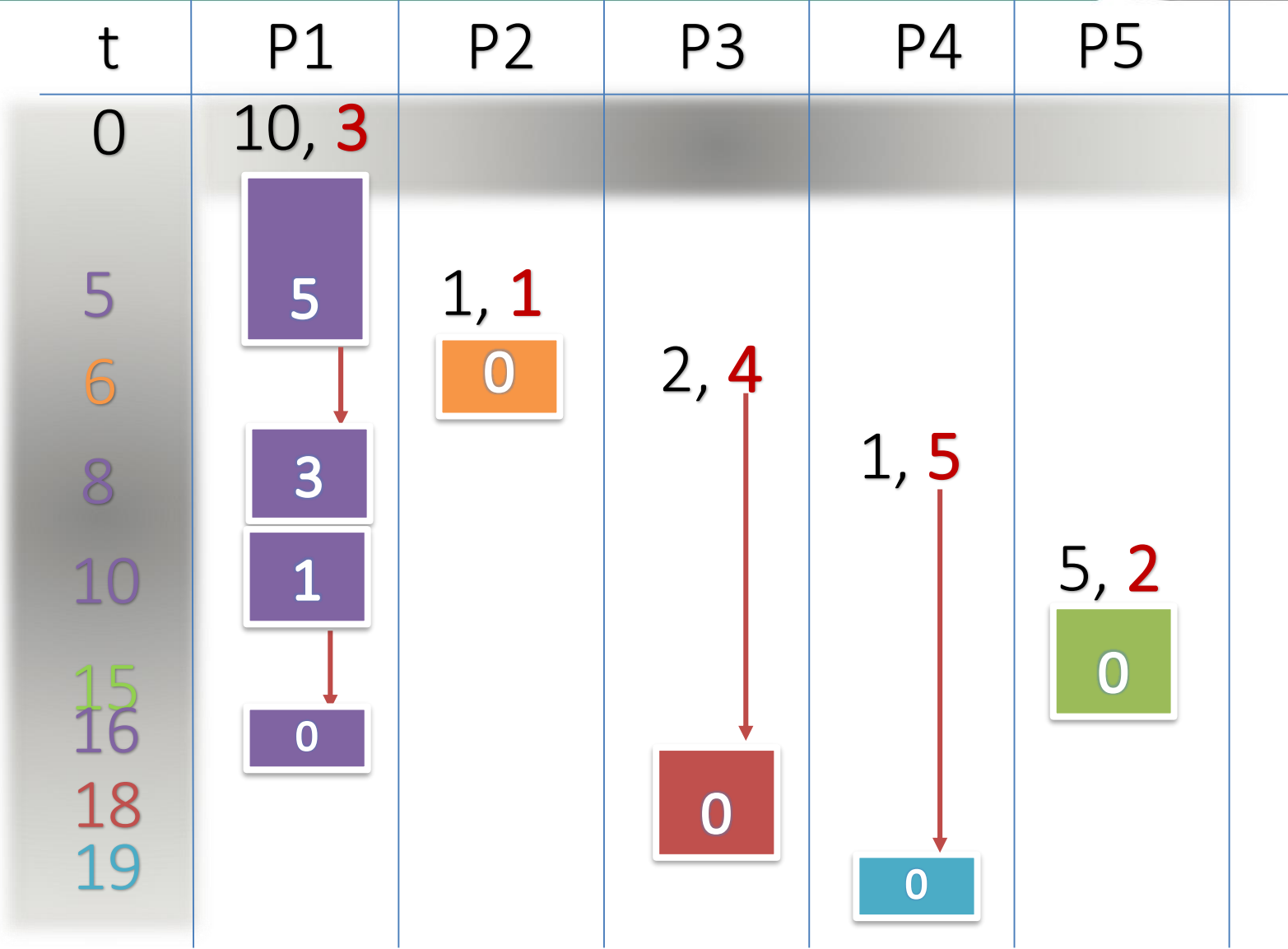
Ví dụ 2: trường hợp không độc quyền

<u>Tiến trình</u>	<u>Thời điểm đến</u>	<u>Thời gian bùong nổ</u>	<u>Độ ưu tiên</u>
P_1	0.0	10	3
P_2	5.0	1	1
P_3	6.0	2	4
P_4	8.0	1	5
P_5	10.0	5	2

Priority Scheduling



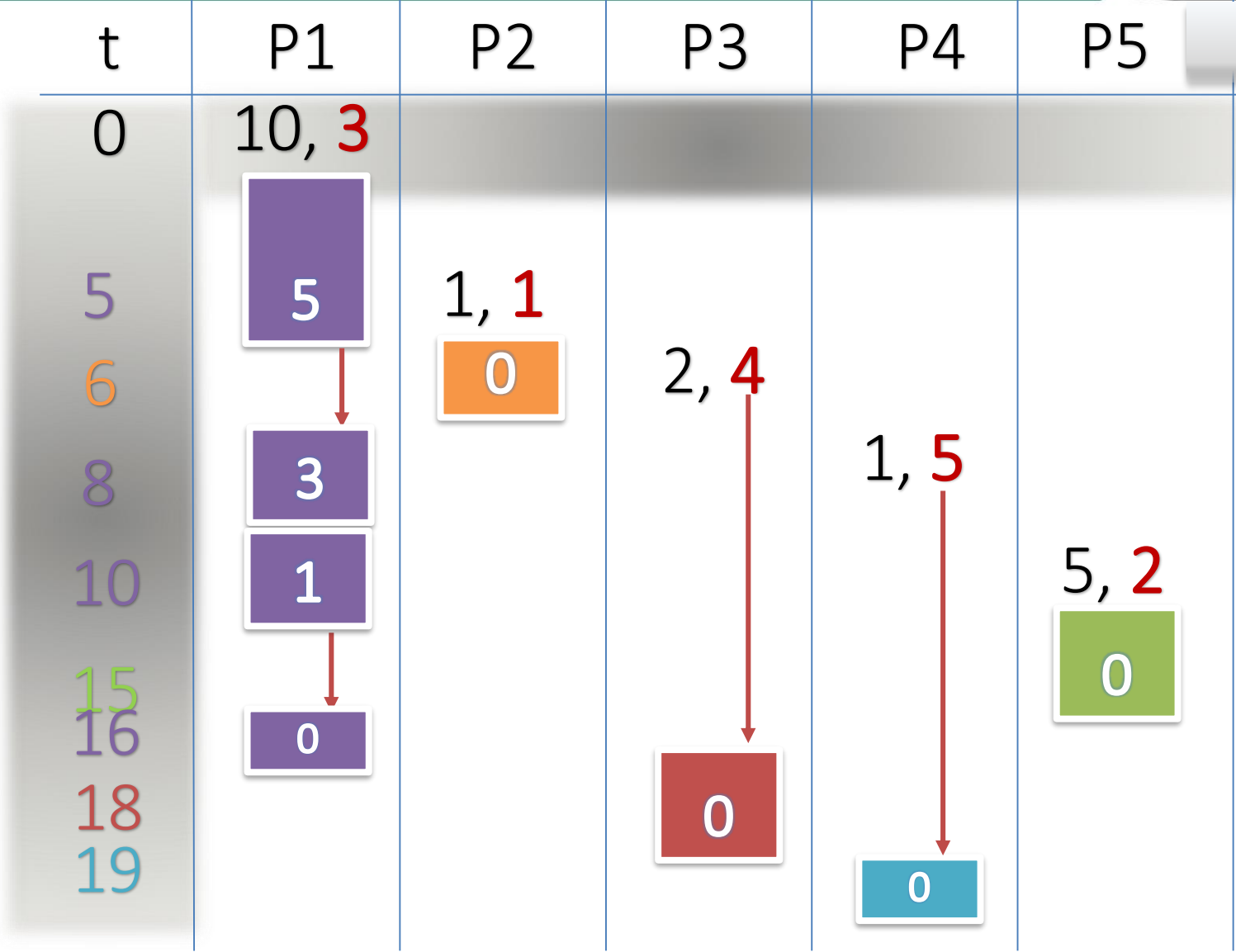
Ví dụ 2: trường hợp không độc quyền



Priority Scheduling



Ví dụ 2: trường hợp không độc quyền

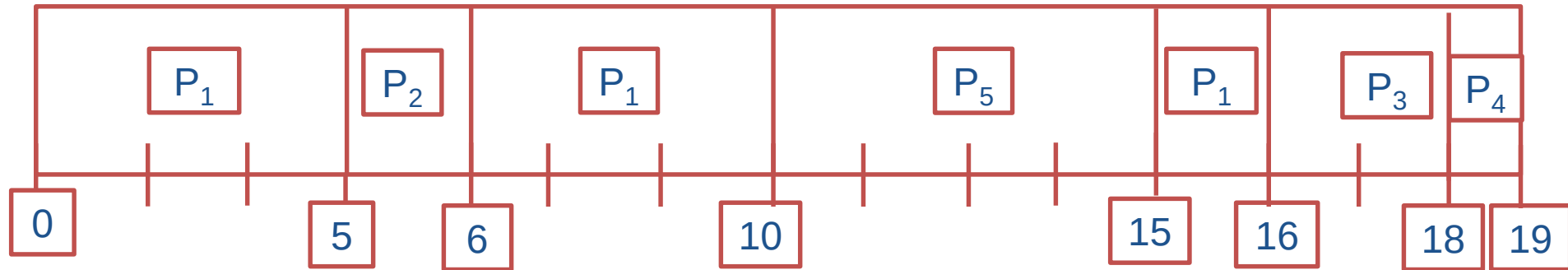


Minh họa lần lượt

Priority Scheduling



Biểu đồ Gantt



➤ Thời gian chờ trung bình: $(6+0+10+10+0)/5=5.2$



Luân phiên (Round Robin Scheduling)



RR-Round Robin



- Được thiết kế đặc biệt cho hệ thống chia sẻ thời gian.
- Hàng đợi sẵn sàng được xem như một hàng đợi vòng.
- Bộ định thời CPU di chuyển vòng quanh hàng đợi sẵn sàng, cấp phát CPU tới mỗi quá trình có khoảng thời gian tối đa bằng một định mức thời gian (quantum).
- Định mức thời gian thường từ 10 đến 100 mili giây.

RR-Round Robin



Ví dụ: *Quantum* = 20

Tiến trình

P_1

P_2

P_3

P_4

Thời gian bùng nổ

53

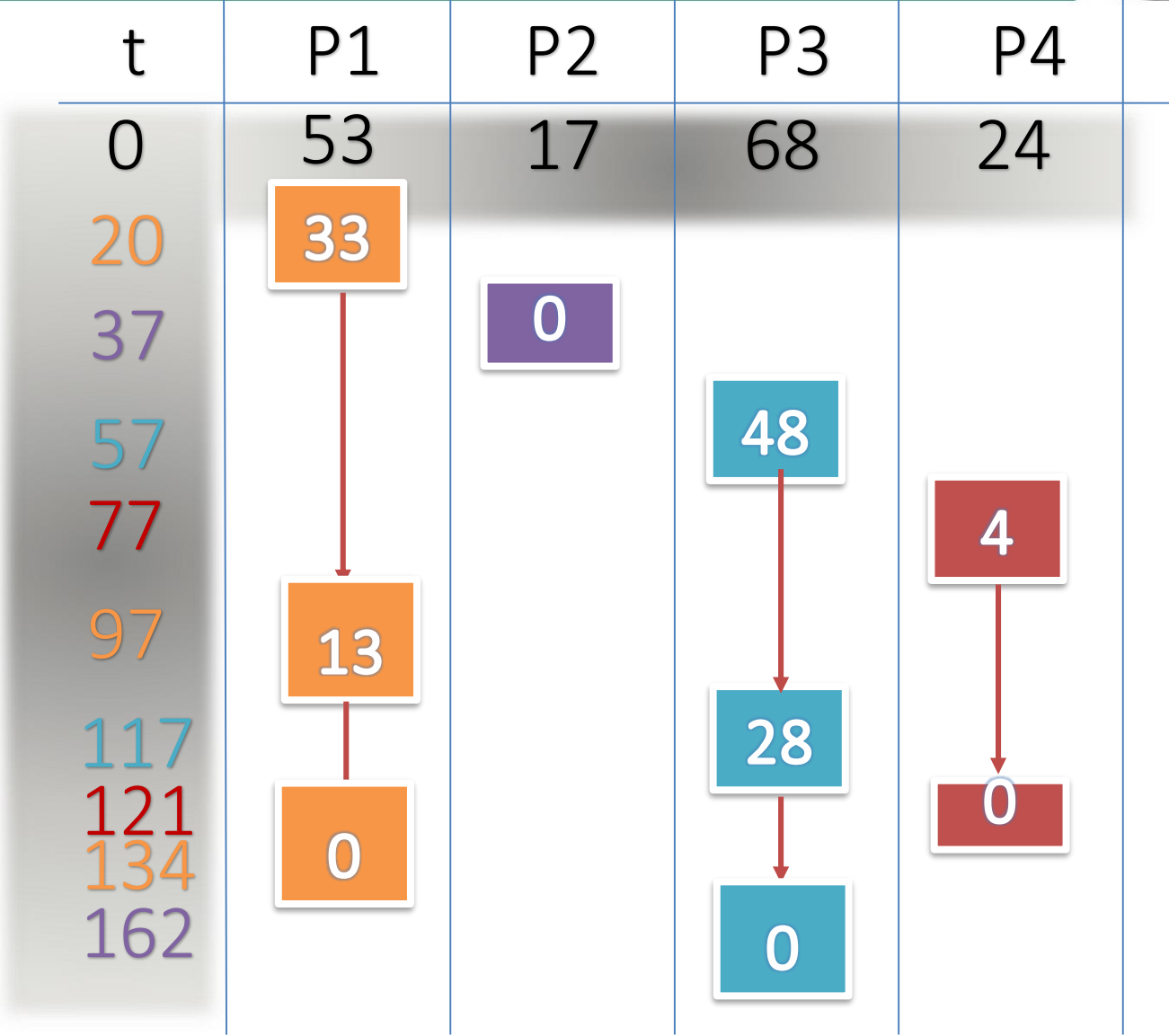
17

68

24

RR-Round Robin

Quantumn = 20

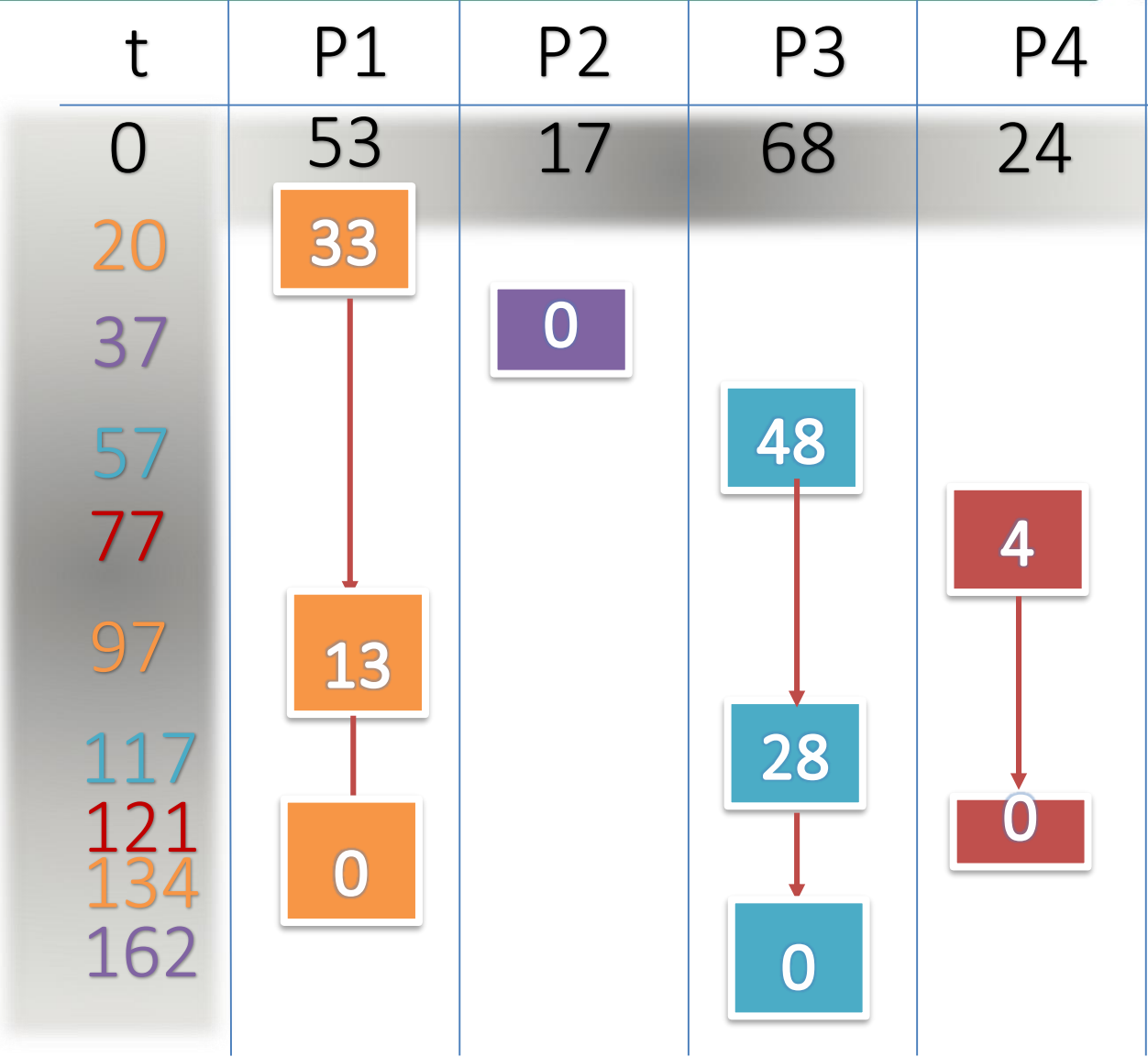


RR-Round Robin



Quantumn = 20

Minh họa lần lượt

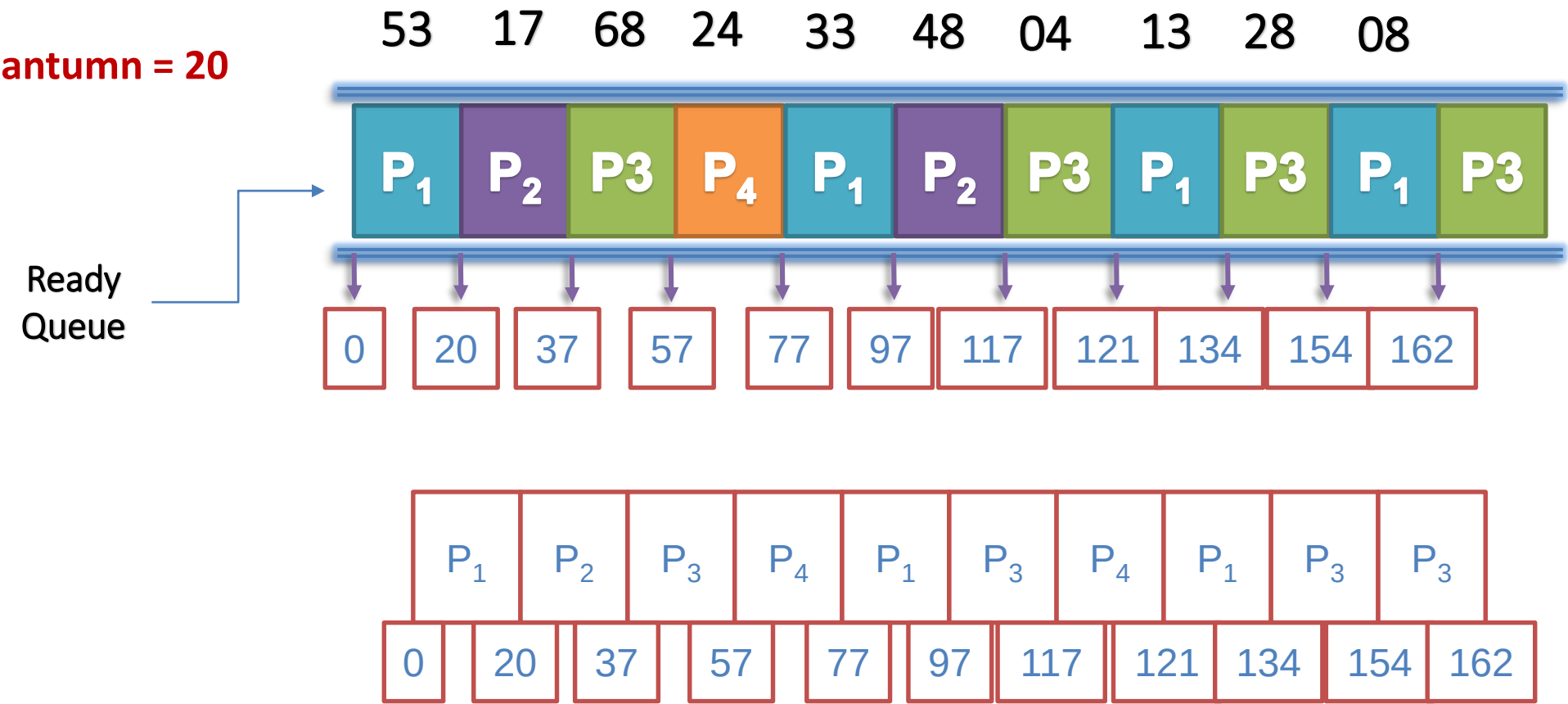


RR-Round Robin



Biểu đồ Gantt là:

Quantum = 20



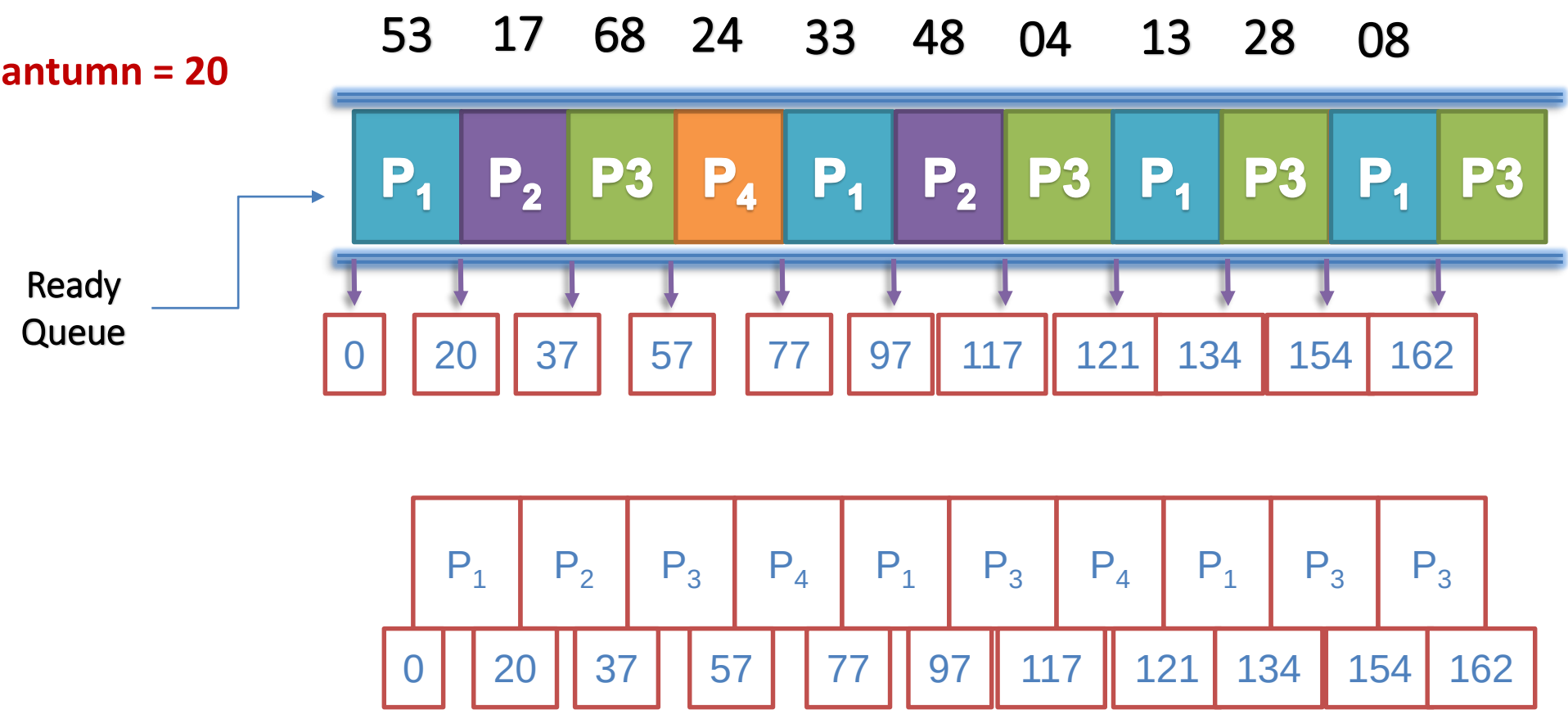
RR-Round Robin



Minh họa lần lượt

Biểu đồ Gantt là:

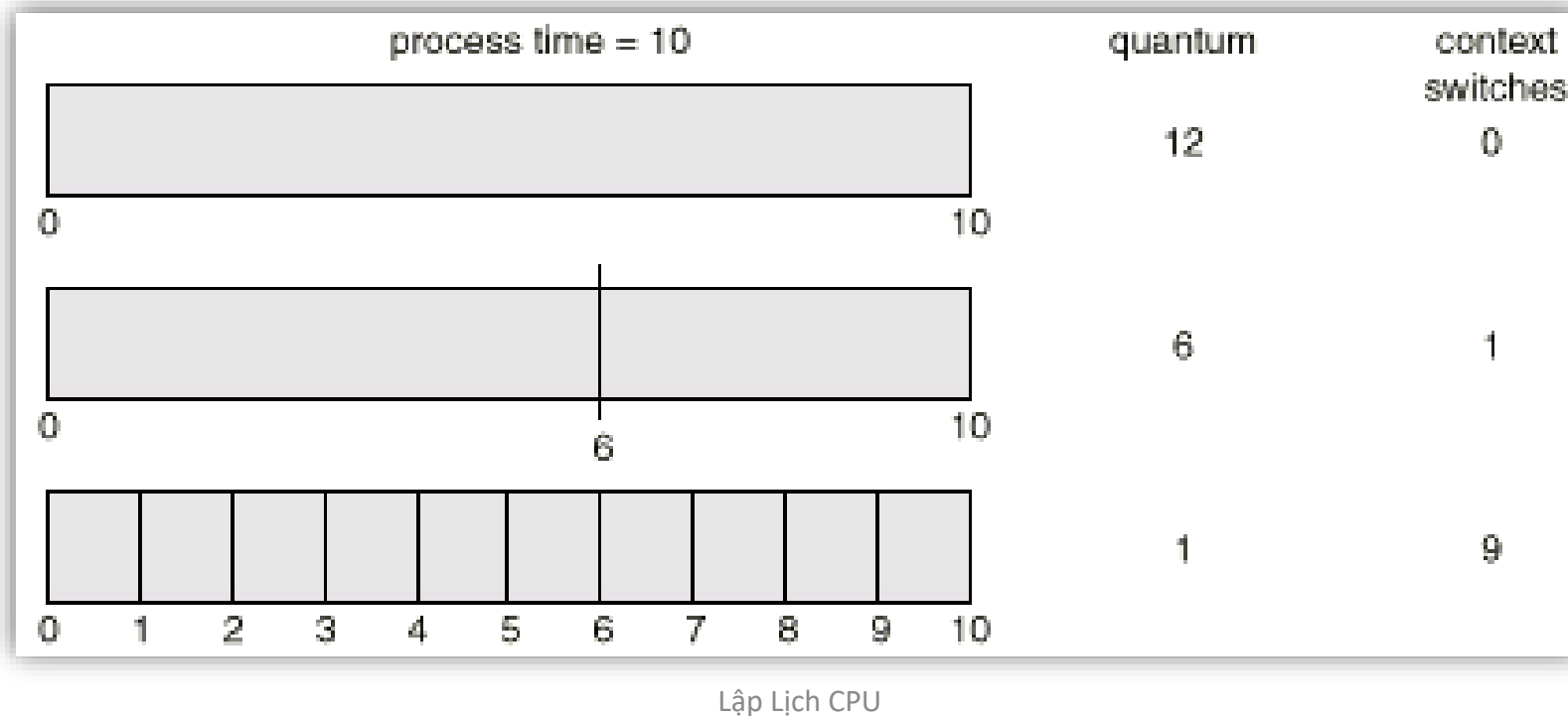
Quantum = 20



RR-Round Robin



- Năng lực của giải thuật RR phụ thuộc nhiều vào kích thước của định mức thời gian.
- Nếu định mức thời gian rất lớn (lượng vô hạn) thì chính sách RR tương tự như chính sách FCFS.
- Nếu định mức thời gian là rất nhỏ (1 mili giây) thì tiếp cận RR được gọi là **chia sẻ bộ xử lý** và xuất hiện tới người dùng như thể mỗi quá trình trong n quá trình có bộ xử lý riêng của chính nó chạy tại $1/n$ tốc độ của bộ xử lý thật.



Hàng đợi đa cấp (Multilevel Queue Scheduling)



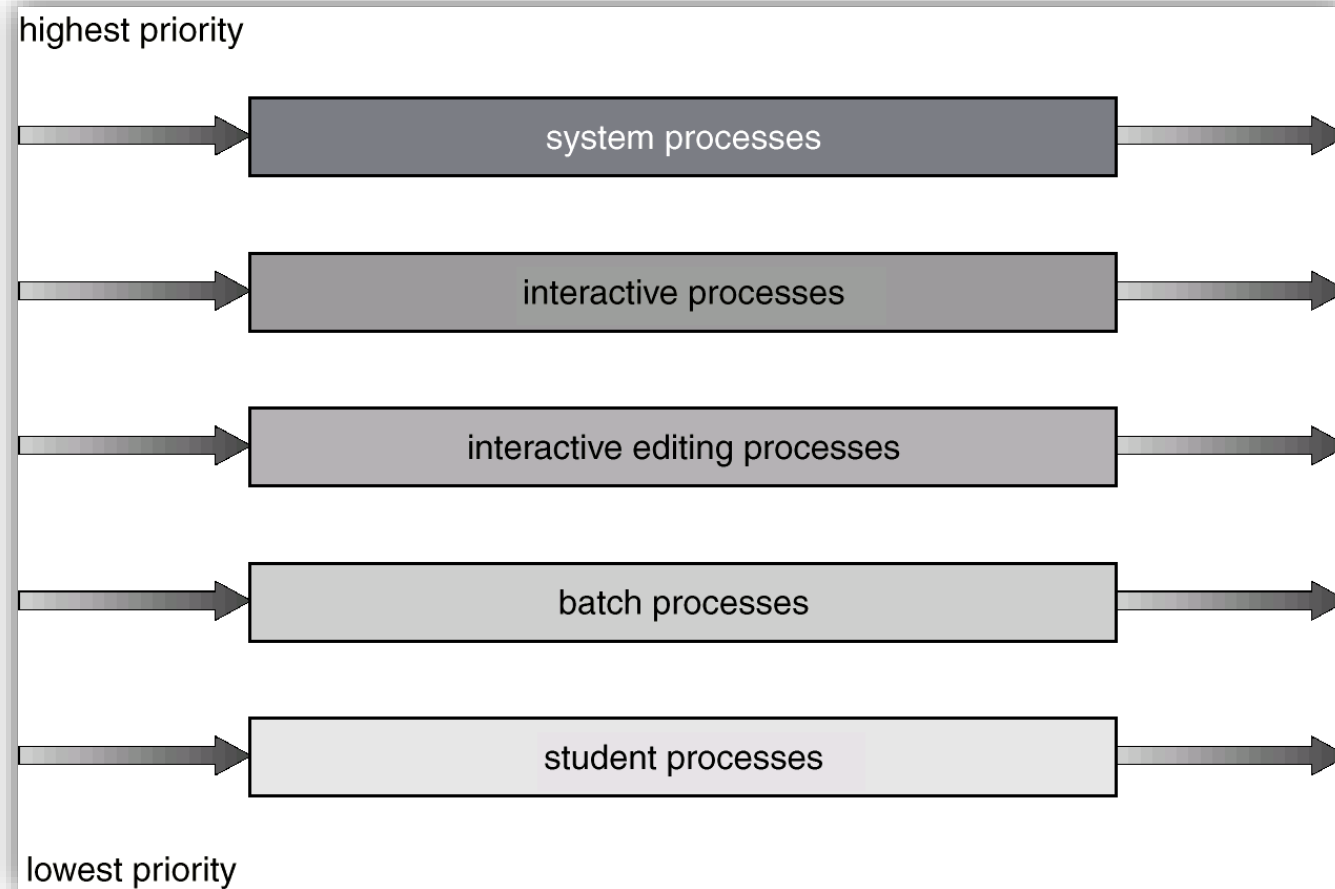
Multilevel Queue



- Một **giải thuật định thời hàng đợi nhiều cấp** (multilevel queue-scheduling algorithm) chia hàng đợi thành nhiều hàng đợi riêng rẽ.
- Các quá trình được gán vĩnh viễn tới một hàng đợi, thường dựa trên thuộc tính của quá trình như kích thước bộ nhớ, độ ưu tiên quá trình hay loại quá trình.
- Mỗi hàng đợi có giải thuật định thời của chính nó.



Multilevel Queue



Multilevel Queue – Ví dụ 1



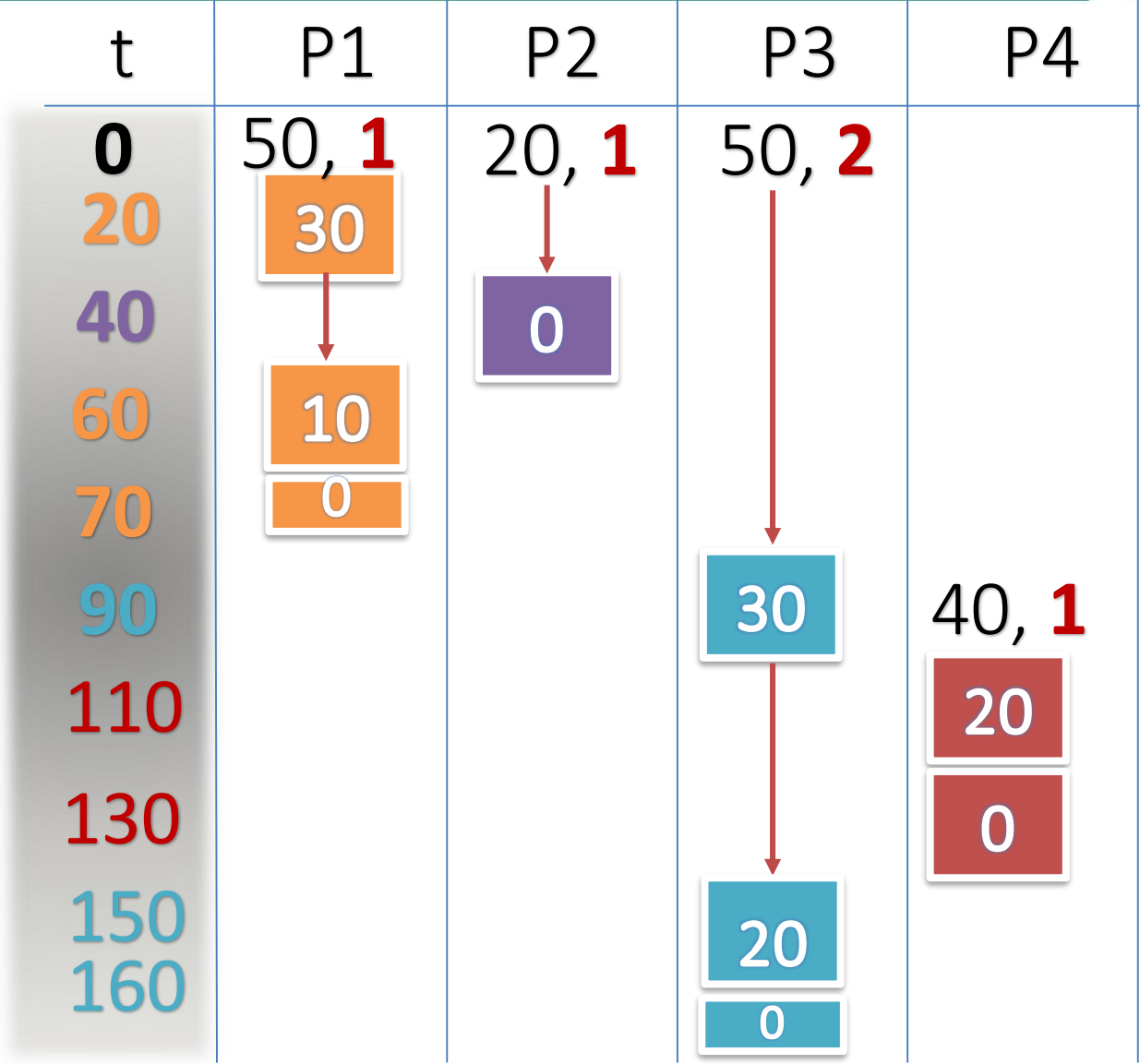
Quantum = 20

<u>Tiến trình</u>	<u>Hàng đợi</u>	<u>Thời điểm đến</u>	<u>Thời gian bùng nổ</u>
P_1	1	0	50
P_2	1	0	20
P_3	2	0	50
P_4	1	90	40

Multilevel Queue – Ví dụ 1



Quantumn = 20

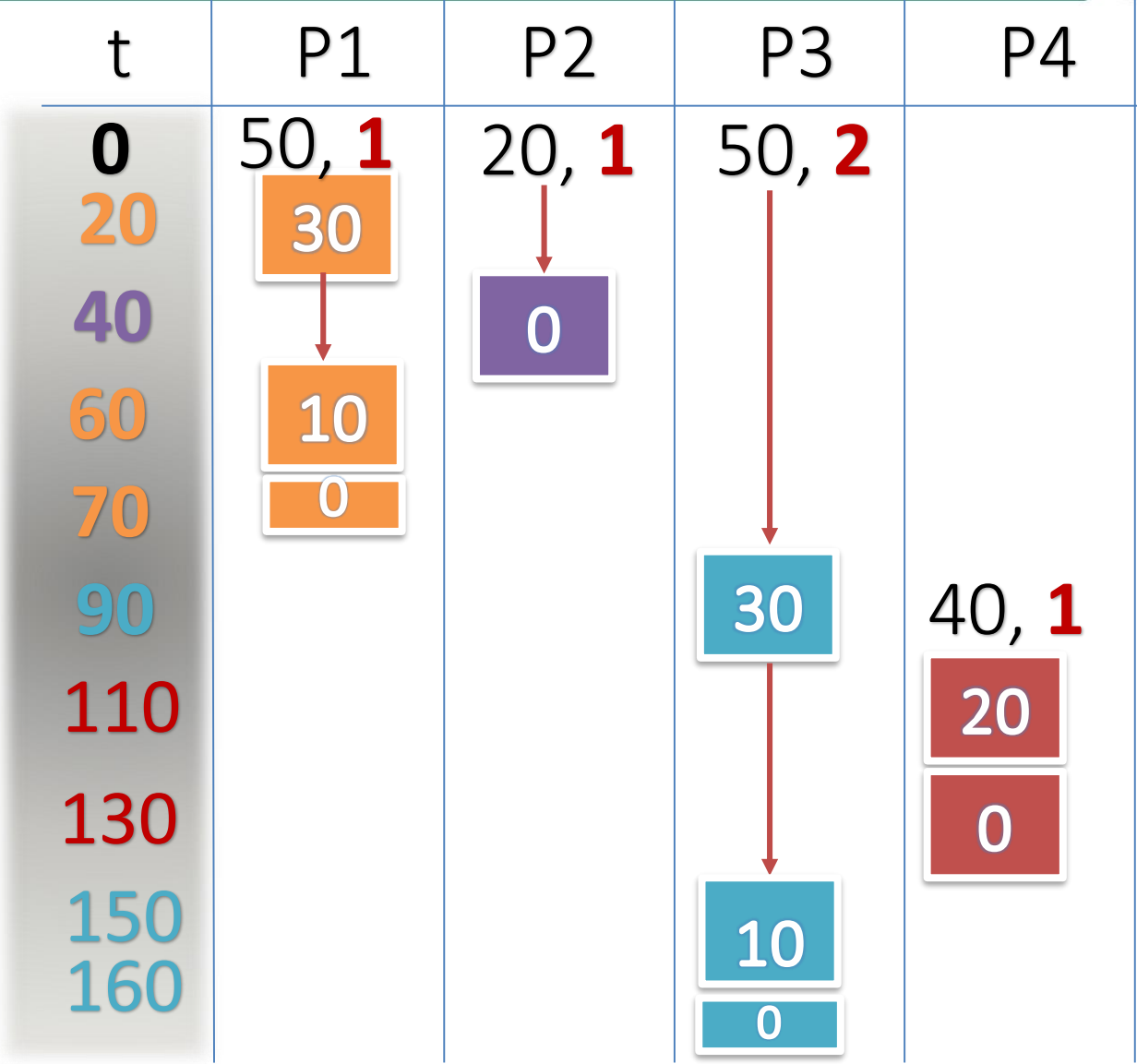


Multilevel Queue – Ví dụ 1



Quantumn = 20

Minh họa lần lượt



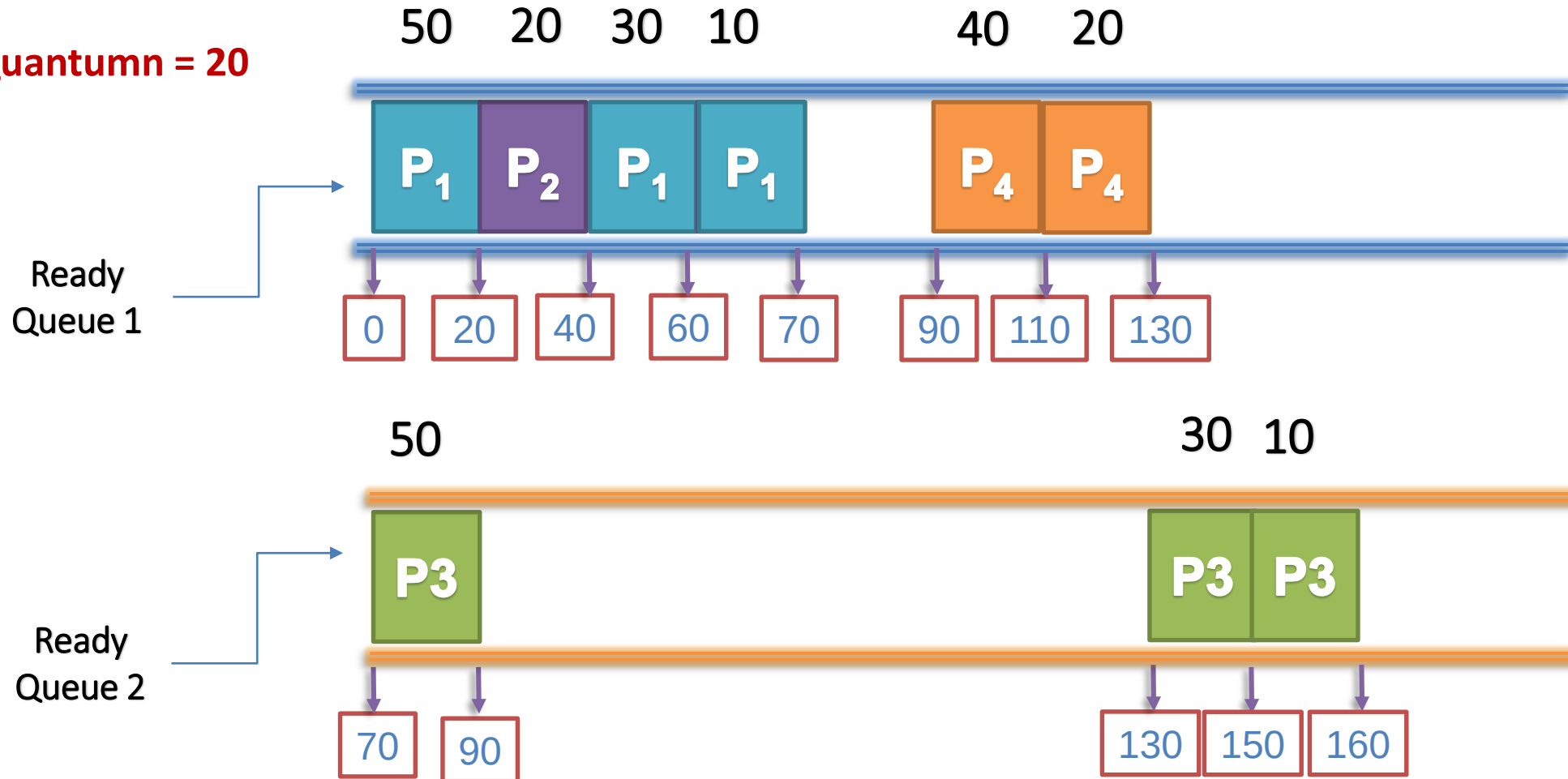
Multilevel Queue – Ví dụ 1



Minh họa lần lượt

Biểu đồ Gantt là:

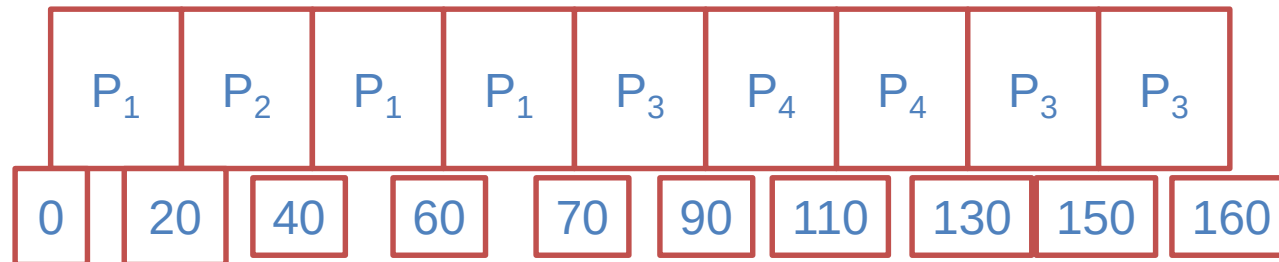
Quantum = 20



Multilevel Queue – Ví dụ 1



Biểu đồ Gantt là:



Thời gian chờ trung bình: $(20+20+110+0)/4=37.5$

Multilevel Queue – Ví dụ 2



Quantum = 20

<u>Tiến trình</u>	<u>Hàng đợi</u>	<u>Thời điểm đến</u>	<u>Thời gian bùng nổ</u>
P_1	1	0	50
P_2	1	40	20
P_3	2	0	50
P_4	1	90	40

Multilevel Queue – Ví dụ 2



Quantumn = 20

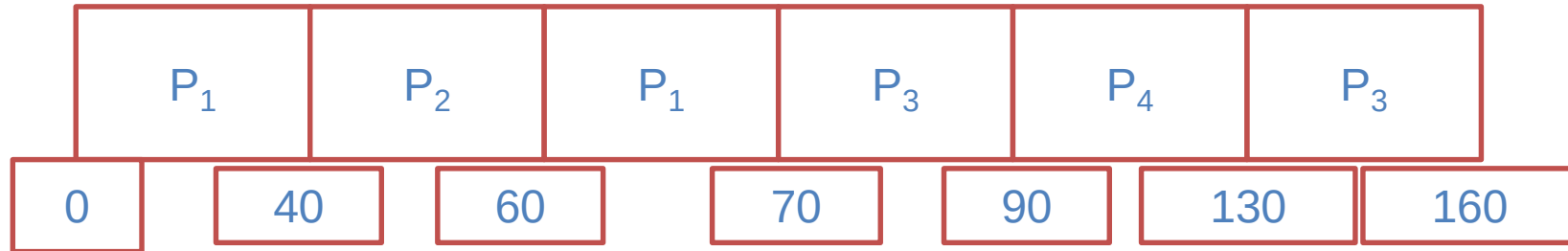
Minh họa lần lượt

t	P1	P2	P3	P4
0	50, 1		50, 2	
20	30			
40	10	20, 1		
60	0	0		
70				
90			30	40, 1
110				20
130				0
150			10	
160			0	

Multilevel Queue – Ví dụ 2



Biểu đồ Gantt là:



Thời gian chờ trung bình: $(20+0+110+0)/4=32.5$

Multilevel Queue – Ví dụ 3



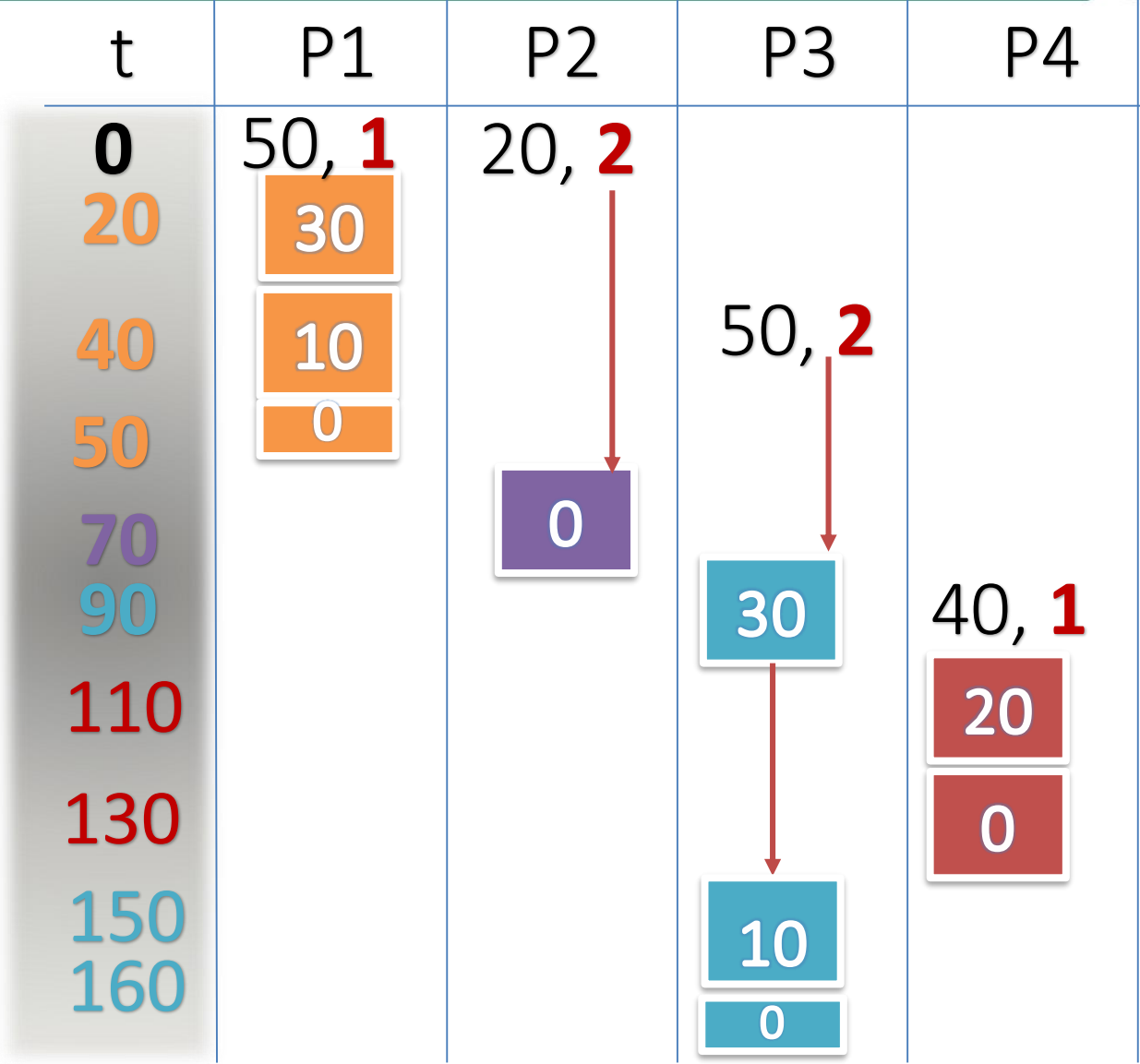
Quantum = 20

<u>Tiến trình</u>	<u>Hàng đợi</u>	<u>Thời điểm đến</u>	<u>Thời gian bùng nổ</u>
P_1	1	0	50
P_2	2	0	20
P_3	2	40	50
P_4	1	90	40

Multilevel Queue – Ví dụ 3



Quantumn = 20

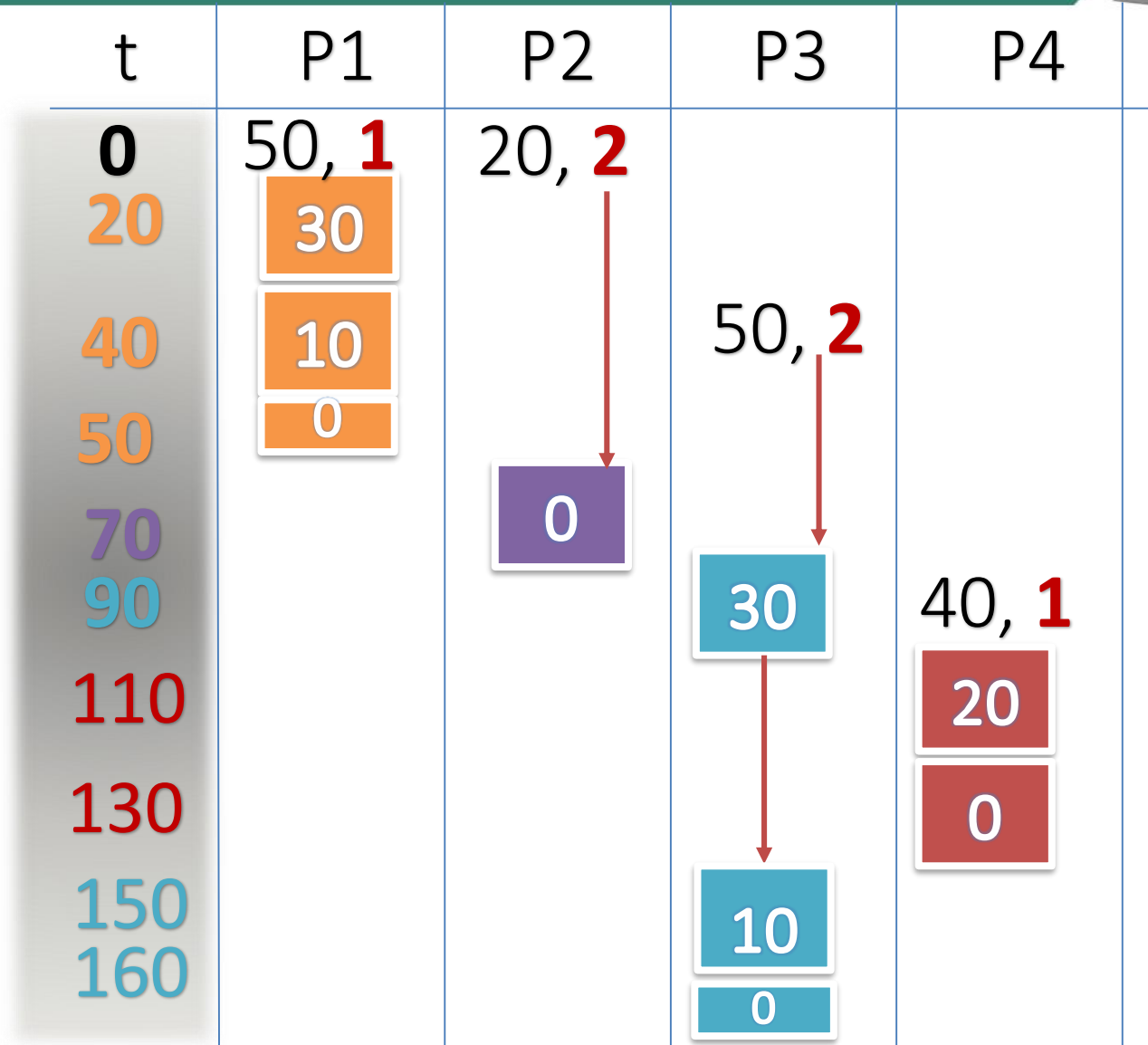


Multilevel Queue – Ví dụ 3



Quantum = 20

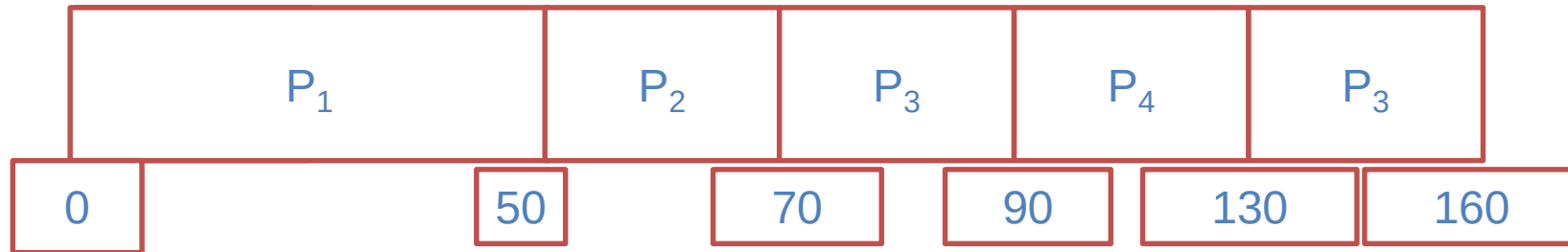
Minh họa lần lượt



Multilevel Queue – Ví dụ 3



Biểu đồ Gantt là:



Thời gian chờ trung bình: $(0+50+70+0)/4=30$

Multilevel Queue – Ví dụ 4

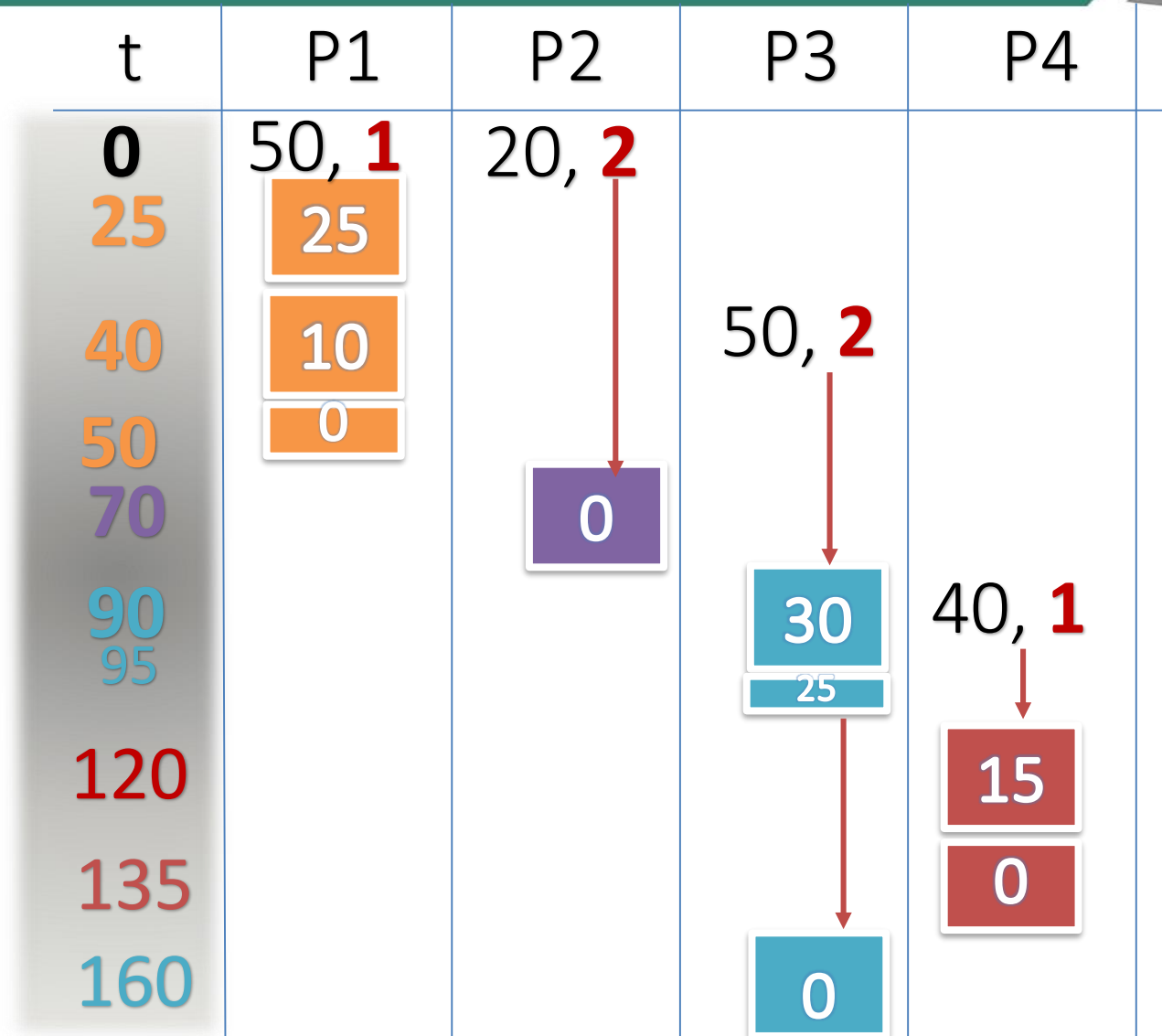


Quantum = 25

<u>Tiến trình</u>	<u>Hàng đợi</u>	<u>Thời điểm đến</u>	<u>Thời gian bùng nổ</u>
P_1	1	0	50
P_2	2	0	20
P_3	2	40	50
P_4	1	90	40

Multilevel Queue – Ví dụ 4

Quantum = 25

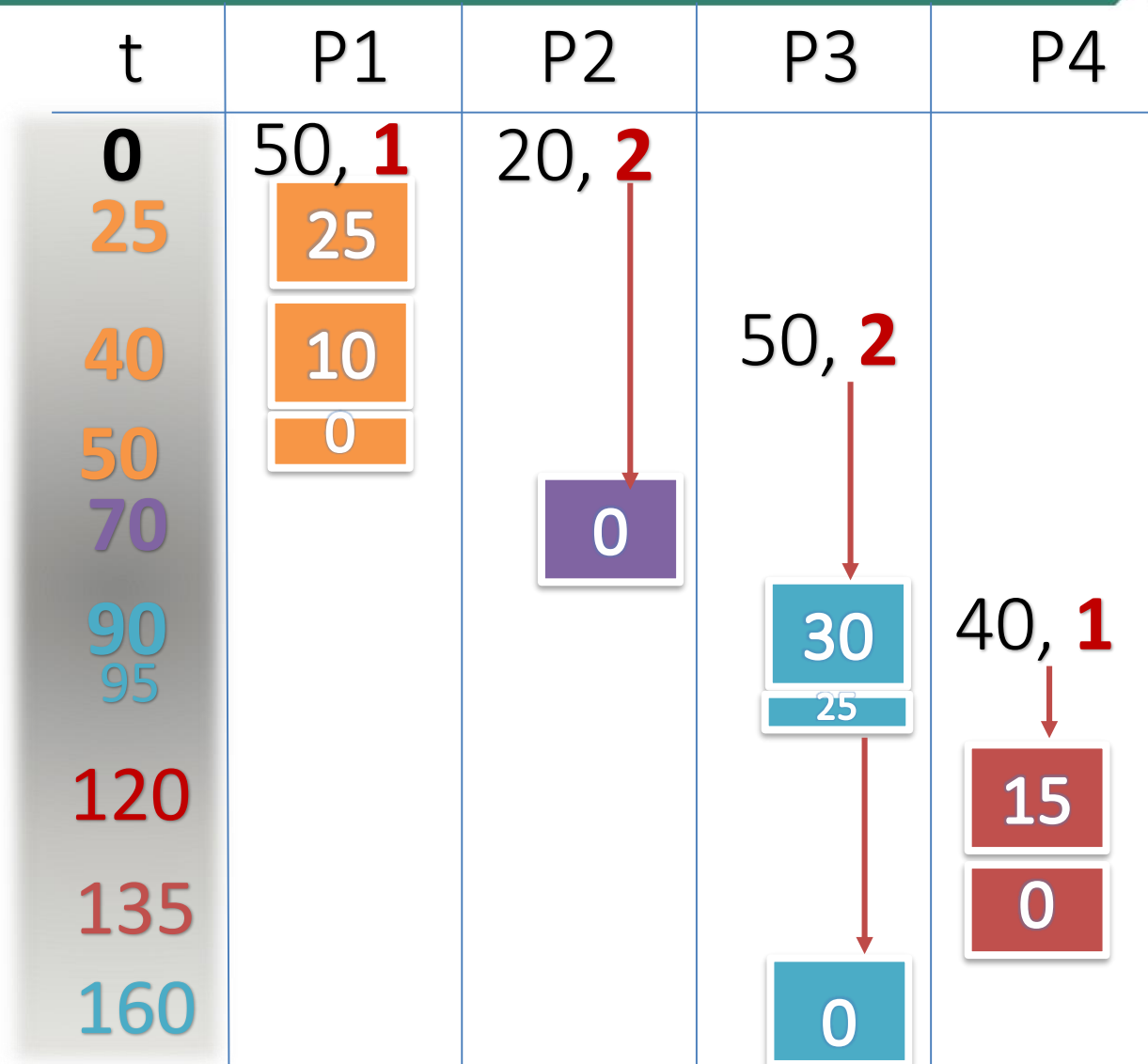


Multilevel Queue – Ví dụ 4



Quantum = 25

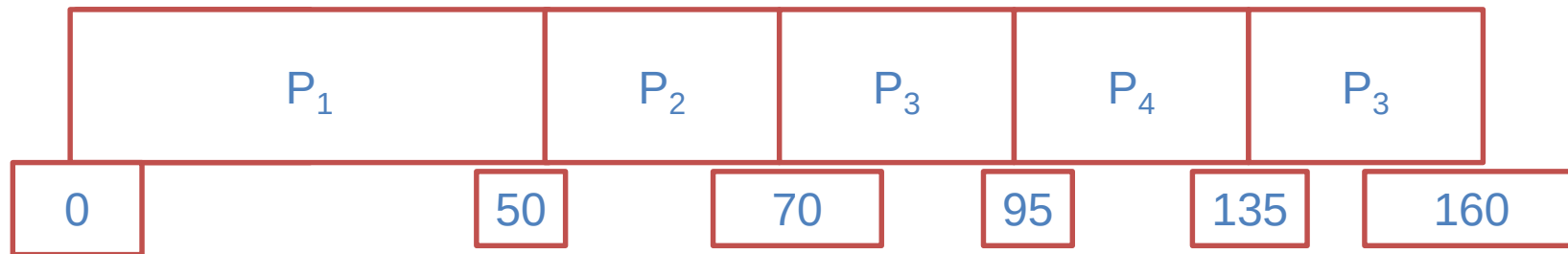
Minh họa lần lượt



Multilevel Queue – Ví dụ 4



Biểu đồ Gantt là:



Thời gian chờ trung bình: $(0+50+70+5)/4=31.25$



Hàng đợi phản hồi đa cấp (Multilevel Feedback Queue)

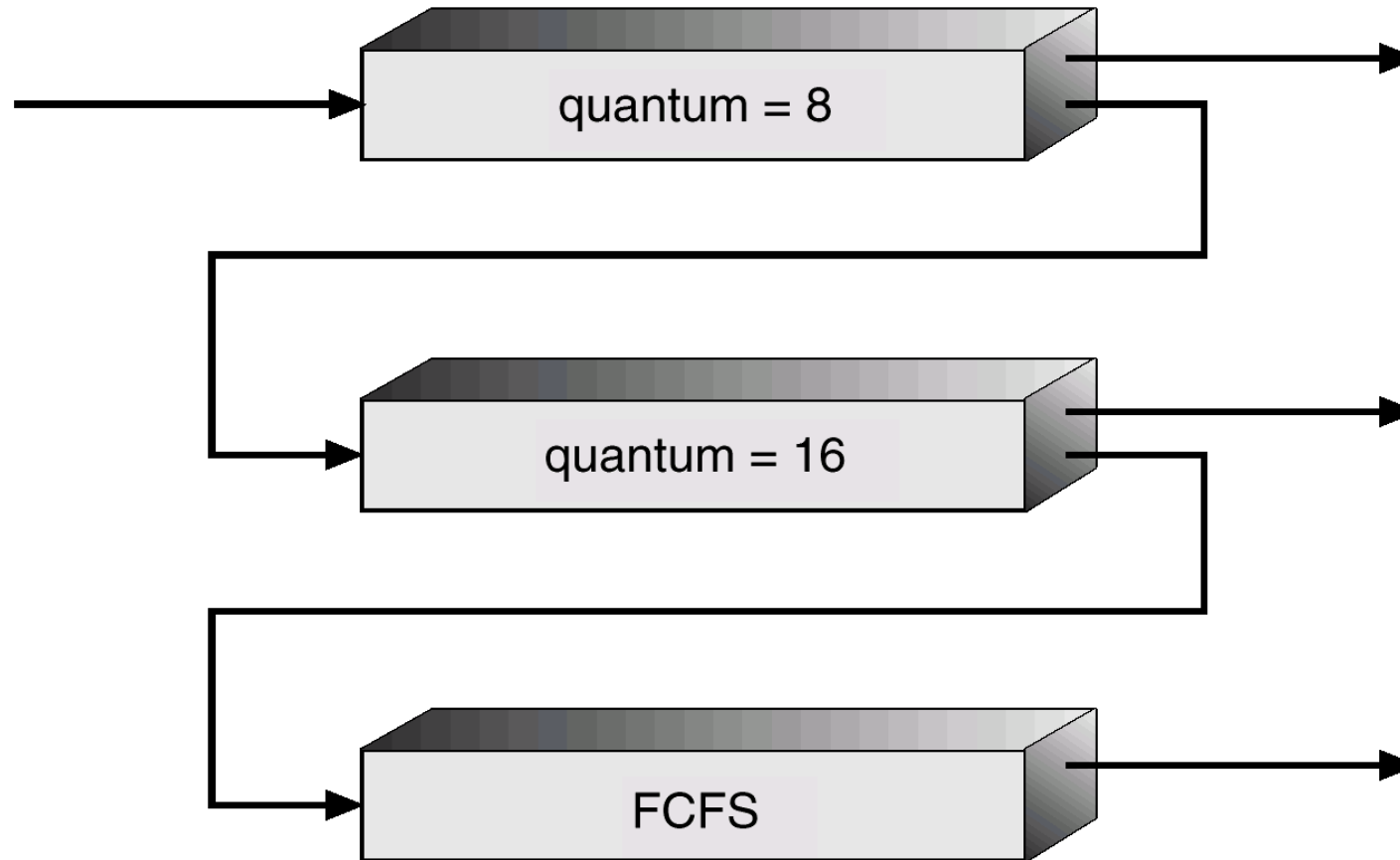


Multilevel Feedback Queue



- **Hàng đợi phản hồi đa cấp** (multilevel feedback queue scheduling) cho phép một quá trình di chuyển giữa các hàng đợi.
- Ý tưởng là tách riêng các quá trình với các đặc điểm chu kỳ CPU khác nhau.
- Nếu một quá trình dùng quá nhiều thời gian CPU thì nó sẽ được di chuyển tới hàng đợi có độ ưu tiên thấp.
- Một quá trình chờ quá lâu trong hàng đợi có độ ưu tiên thấp hơn có thể được di chuyển tới hàng đợi có độ ưu tiên cao hơn.

Multilevel Feedback Queue



Multilevel Feedback Queue – Ví dụ 1



Quantum 1 = 8

Quantum 2 = 16

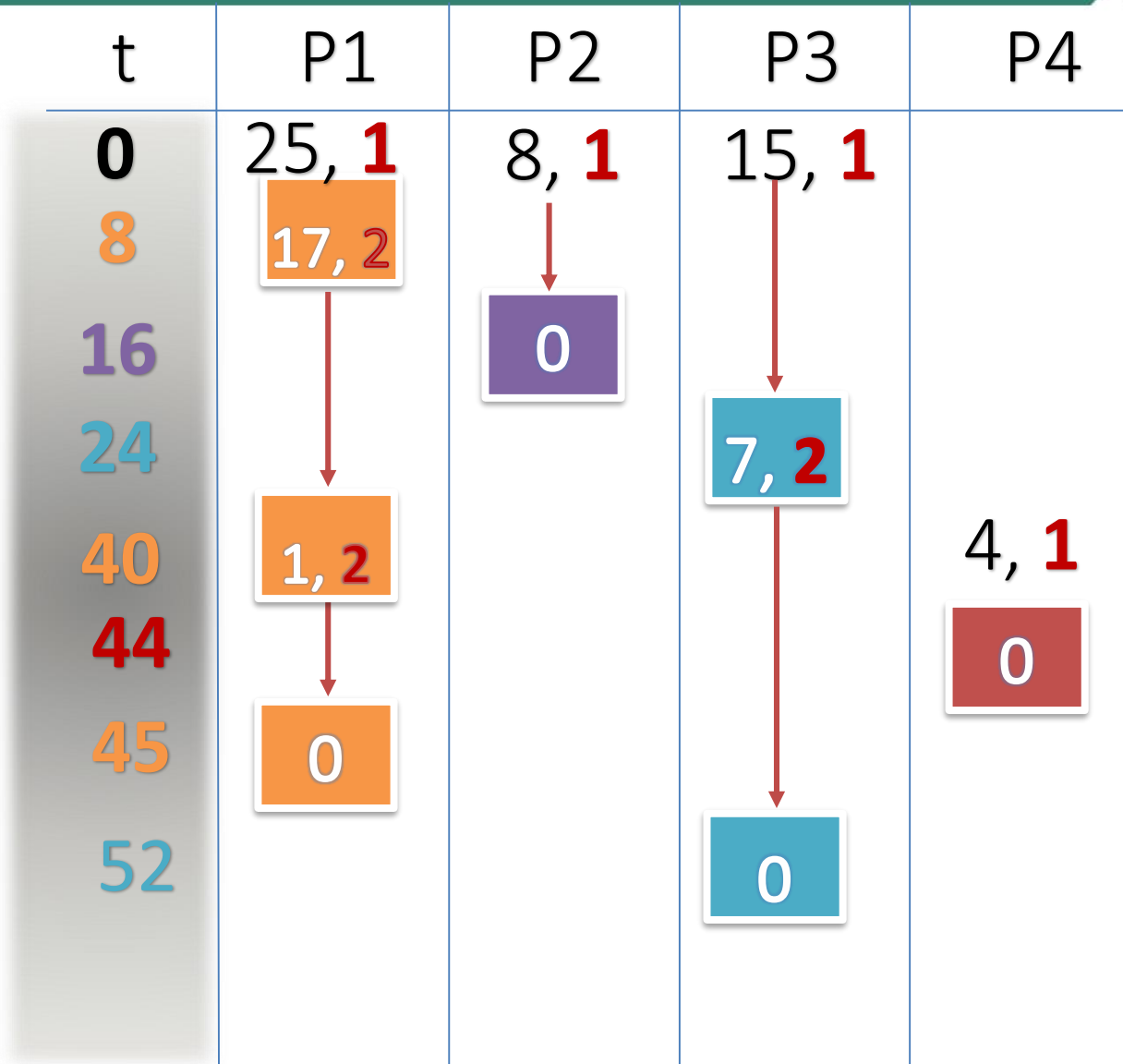
<u>Tiến trình</u>	<u>Thời điểm đến</u>	<u>Thời gian bùng nổ</u>
P_1	0	25
P_2	0	8
P_3	0	15
P_4	40	4

Multilevel Feedback Queue – Ví dụ 1



Quantum 1 = 8
Quantum 2 = 16

Minh họa lần lượt



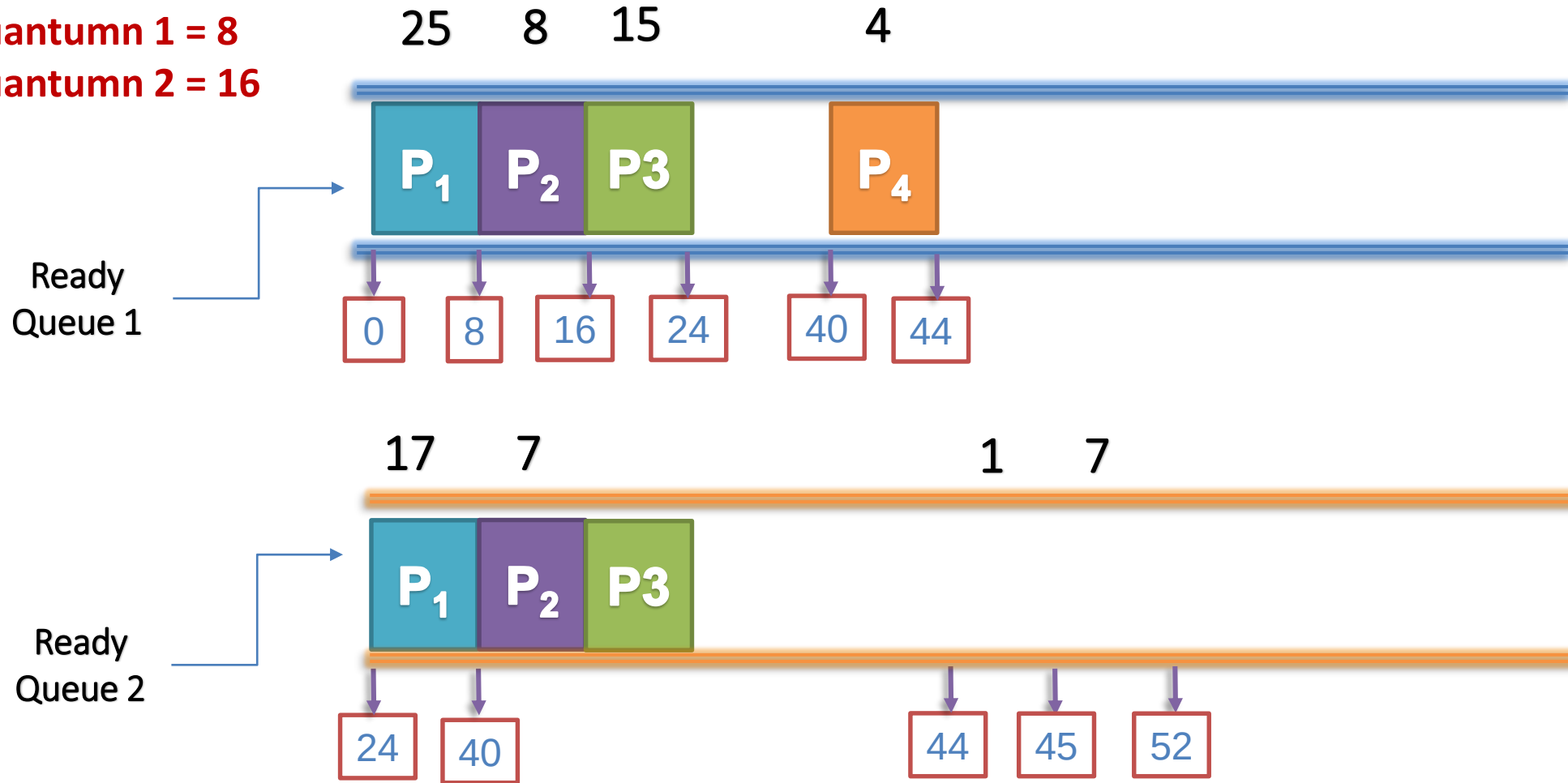
Multilevel Feedback Queue – Ví dụ 1



Minh họa lần lượt

Biểu đồ Gantt là:

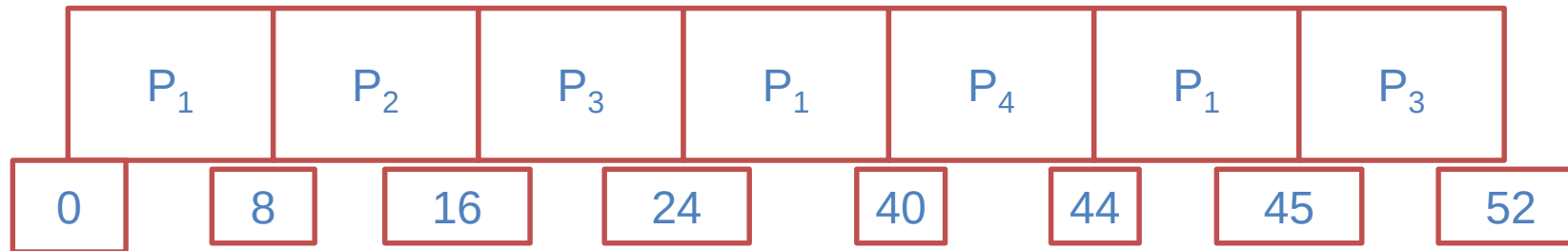
Quantum 1 = 8
Quantum 2 = 16



Multilevel Feedback Queue – Ví dụ 1



Biểu đồ Gantt là:



Thời gian chờ trung bình: $(20+8+37+0)/4=16.25$

Multilevel Feedback Queue – Ví dụ 2



Quantum 1 = 8

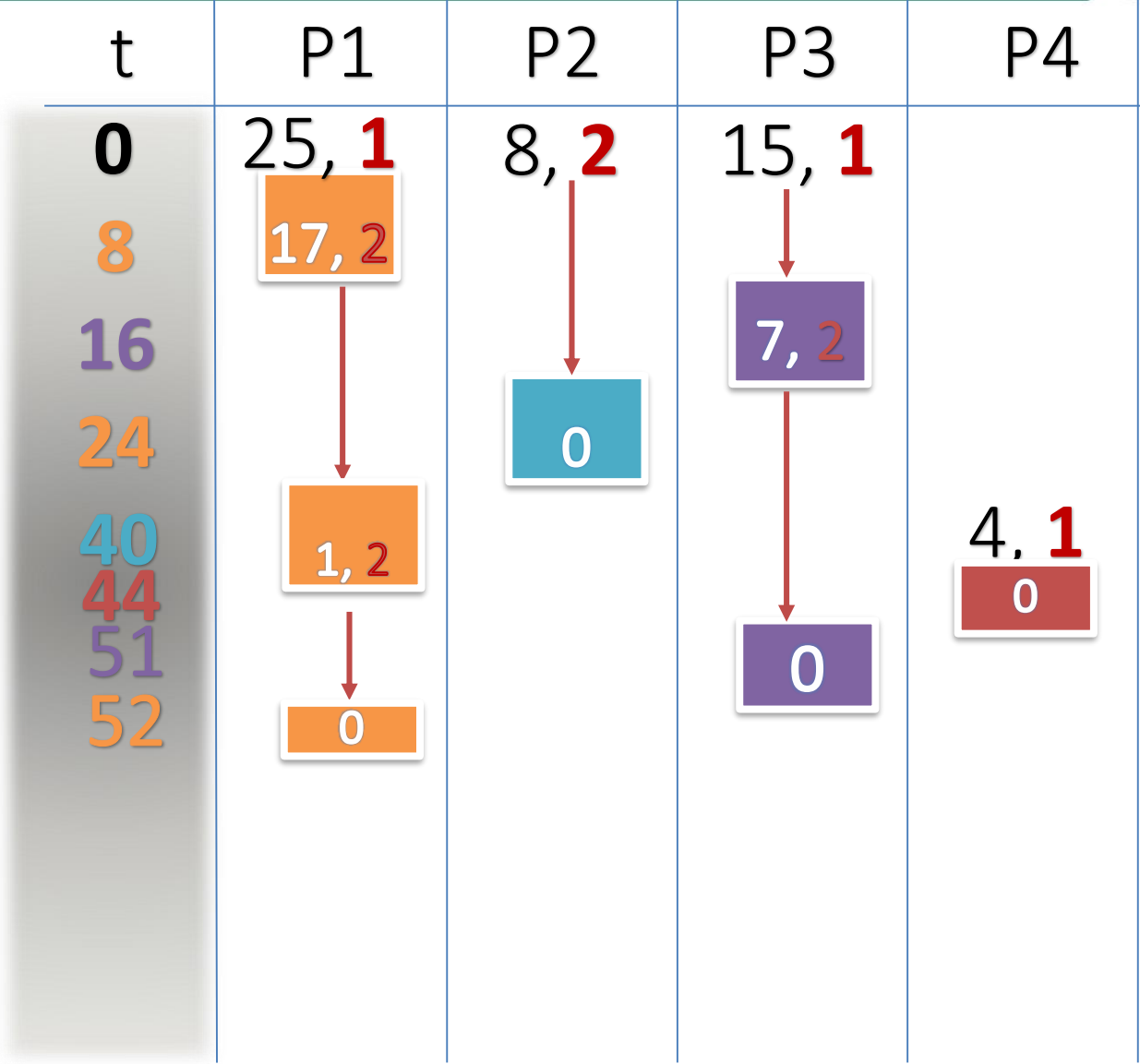
Quantum 2 = 16

<u>Tiến trình</u>	<u>Hàng đợi</u>	<u>Thời điểm đến</u>	<u>Thời gian bùng nổ</u>
P_1	1	0	25
P_2	2	0	8
P_3	1	0	15
P_4	1	40	4

Multilevel Feedback Queue – Ví dụ 2



Quantum 1 = 8
Quantum 2 = 16



Minh họa lần lượt

LUYỆN TẬP



Luyện tập



Bài 1: Thực hiện điều phối CPU với giải thuật **FCFS**

Tiến trình	Thời gian đến	Thời gian thực hiện
P1	0	4
P2	3	3
P3	1	3
P4	5	3
P5	2	1

Luyện tập



Bài 2: Thực hiện điều phối CPU với giải thuật **SJF**

Tiến trình	Thời gian đến	Thời gian thực hiện
P1	0	4
P2	3	3
P3	1	3
P4	5	3
P5	2	1

Luyện tập



Bài 3: Thực hiện điều phối CPU với giải thuật **Priority**

Tiến trình	Thời gian đến	Thời gian thực hiện	Độ ưu tiên
P1	0	11	2
P2	3	7	3
P3	5	4	1
P4	5	2	3
P5	2	8	2

Luyện tập



Bài 4: Thực hiện điều phối CPU với giải thuật **RR**

Quantum=3		
Tiến trình	Thời gian đến	Thời gian thực hiện
P1	0	11
P2	3	7
P3	5	4
P4	5	2
P5	2	8

So sánh các giải thuật lập lịch CPU



So sánh các giải thuật



- **FCFS:** Tiến trình nào vào hàng đợi sẵn sàng trước được CPU xử lý trước
- Nhược điểm:
 - Là gì?
 - Khắc phục: Sử dụng phương pháp nào để hỗ trợ cho nhược điểm của giải thuật này?

So sánh các giải thuật



- **SJF:** Tiến trình được lập lịch theo thời gian bùng nổ của tiến trình
- Nhược điểm:
 - Là gì?
 - Khắc phục: Sử dụng phương pháp nào để hỗ trợ cho nhược điểm của giải thuật này?

So sánh các giải thuật



- **Priority:** Tiến trình được lập lịch theo độ ưu tiên của tiến trình
- Nhược điểm:
 - Là gì?
 - Khắc phục: Sử dụng phương pháp nào để hỗ trợ cho nhược điểm của giải thuật này?

So sánh các giải thuật



- **Round Robin:** Tiến trình được lập lịch theo các khoảng thời gian bằng nhau luân phiên
- Nhược điểm:
 - Là gì?
 - Khắc phục: Sử dụng phương pháp nào để hỗ trợ cho nhược điểm của giải thuật này?

So sánh các giải thuật



- **Multilevel Queue:** Các tiến trình được xếp vào các hàng đợi sẵn sàng khác nhau. Tiến trình được lập lịch theo từng hàng đợi riêng rẽ. Tiến trình nằm trong hàng đợi có độ ưu tiên cao hơn sẽ được CPU xử lý trước.
- Nhược điểm:
 - Là gì?
 - Khắc phục: Sử dụng phương pháp nào để hỗ trợ cho nhược điểm của giải thuật này?

So sánh các giải thuật



- **Multilevel Feedback Queue:** Các tiến trình được xếp vào các hàng đợi sẵn sàng khác nhau. Tiến trình được lập lịch theo từng hàng đợi riêng rẽ. Tiến trình nằm trong hàng đợi có độ ưu tiên cao hơn sẽ được CPU xử lý trước. Tiến trình sau khi được CPU xử lý sẽ đi vào hàng đợi có độ ưu tiên thấp hơn.
- Nhược điểm:
 - Là gì?
 - Khắc phục: Sử dụng phương pháp nào để hỗ trợ cho nhược điểm của giải thuật này?

Bài tập vận dụng



SRT với $T_{\min_CPU}$



Ví dụ 1: $T_{\min_CPU} = 6$

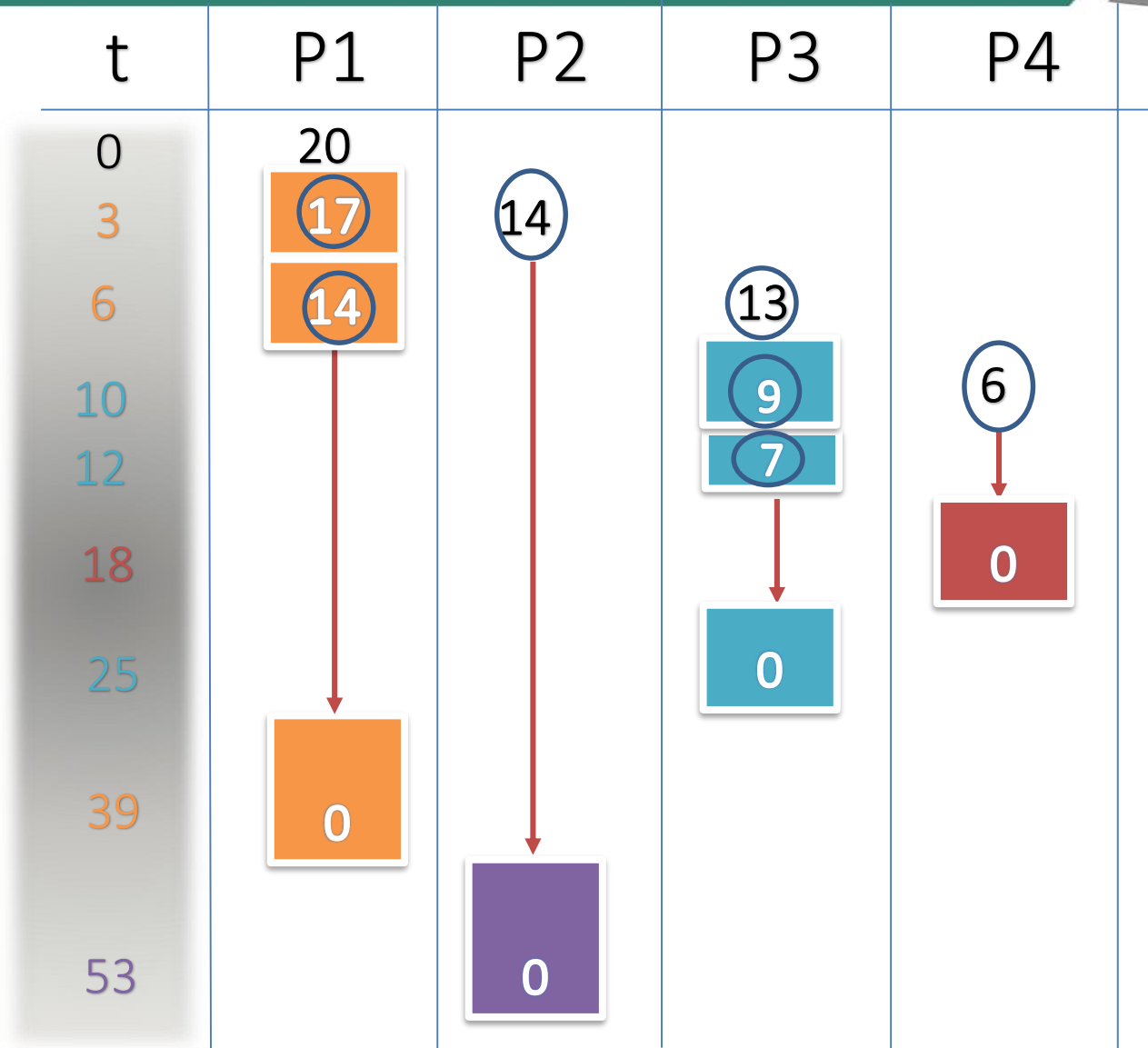
<u>Tiến trình</u>	<u>Thời điểm đến</u>	<u>Thời gian bù đắp</u>
P_1	0.0	20
P_2	3.0	14
P_3	6.0	13
P_4	10.0	6

SRT với $T_{\min_CPU}$



Ví dụ 1: $T_{\min_CPU} = 6$

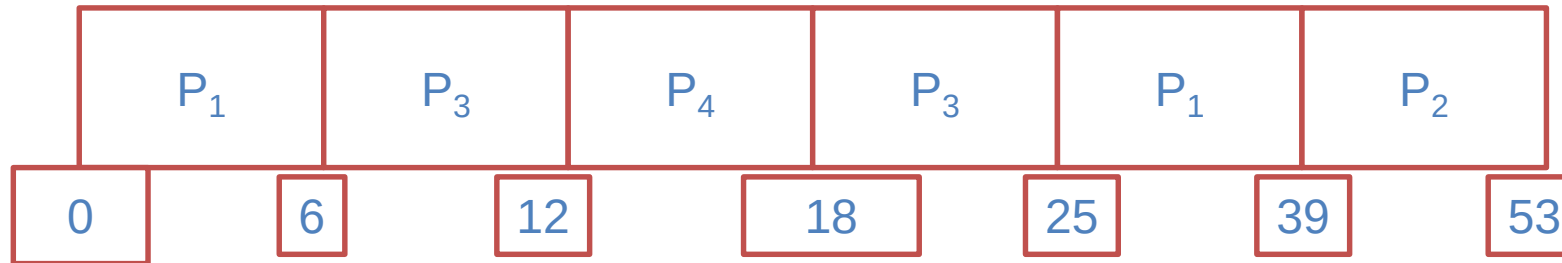
Minh họa lần lượt



Lập Lịch CPU



Biểu đồ Gantt là:



Thời gian chờ trung bình: $(19+36+6+2)/4=15.75$

Priority Scheduling với SJF



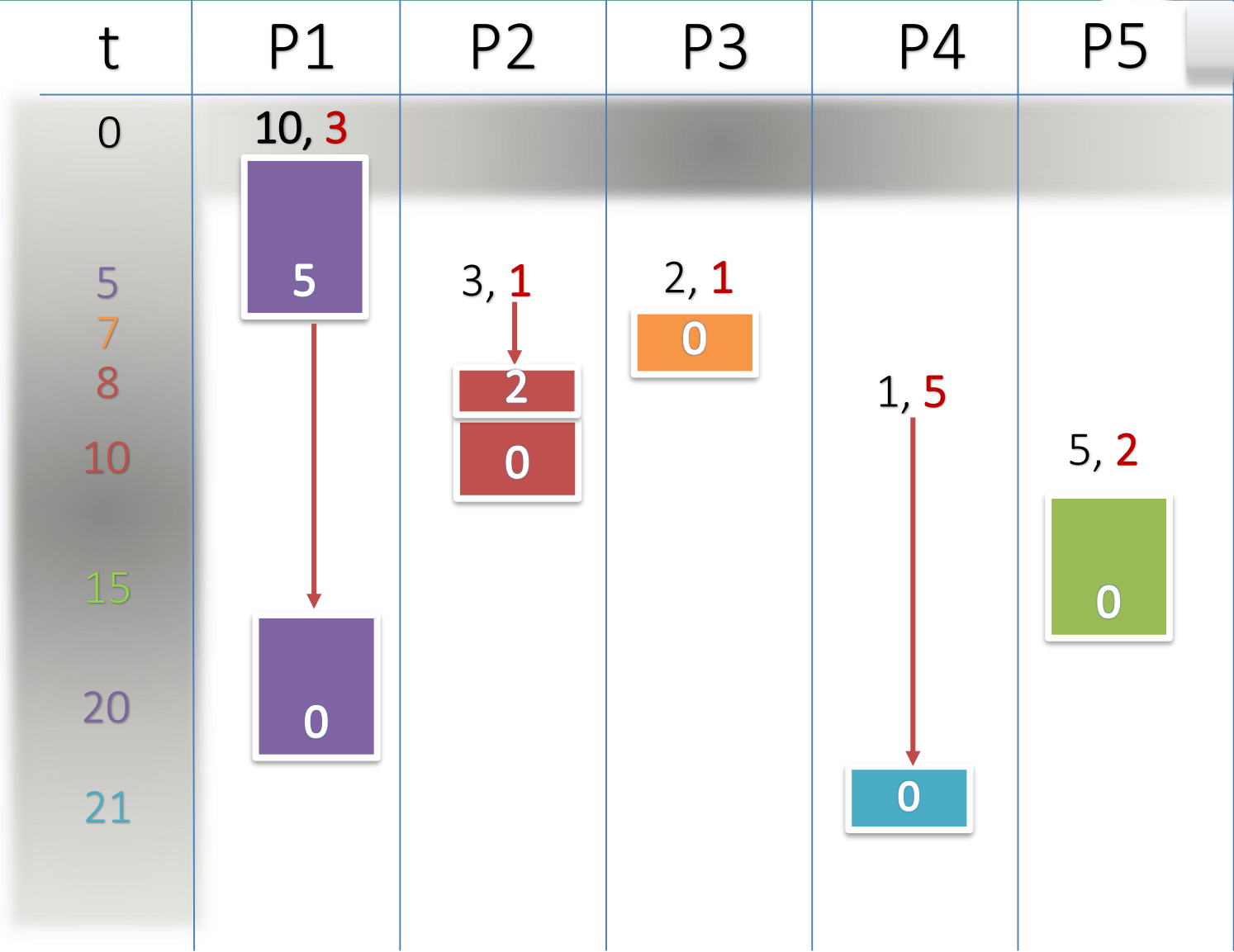
Ví dụ 2: trường hợp không độc quyền

<u>Tiến trình</u>	<u>Thời điểm đến</u>	<u>Thời gian bù đắp</u>	<u>Độ ưu tiên</u>
P_1	0.0	10	3
P_2	5.0	3	1
P_3	5.0	2	1
P_4	8.0	1	5
P_5	10.0	5	2

Priority Scheduling với SJF



Ví dụ 2: trường hợp không độc quyền

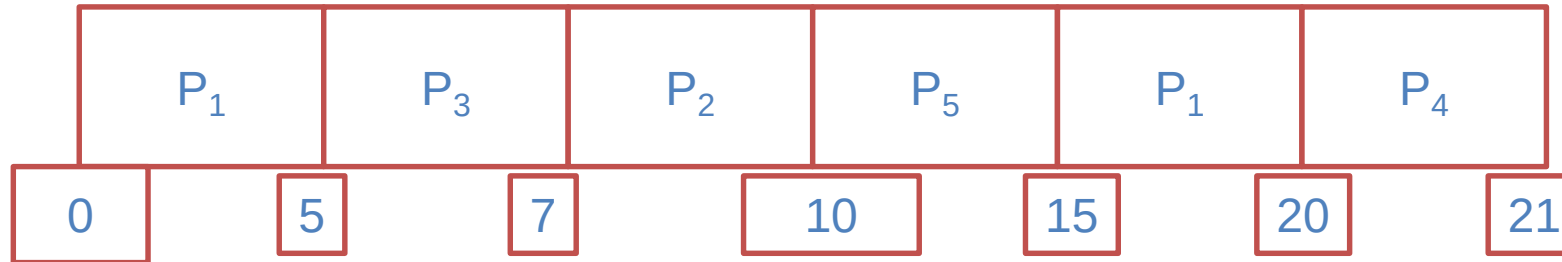


Minh họa lần lượt

Priority Scheduling với SJF



Biểu đồ Gantt là:



Thời gian chờ trung bình: $(10+2+0+12+0)/5=4.8$

Priority Scheduling với độ ưu tiên thay đổi

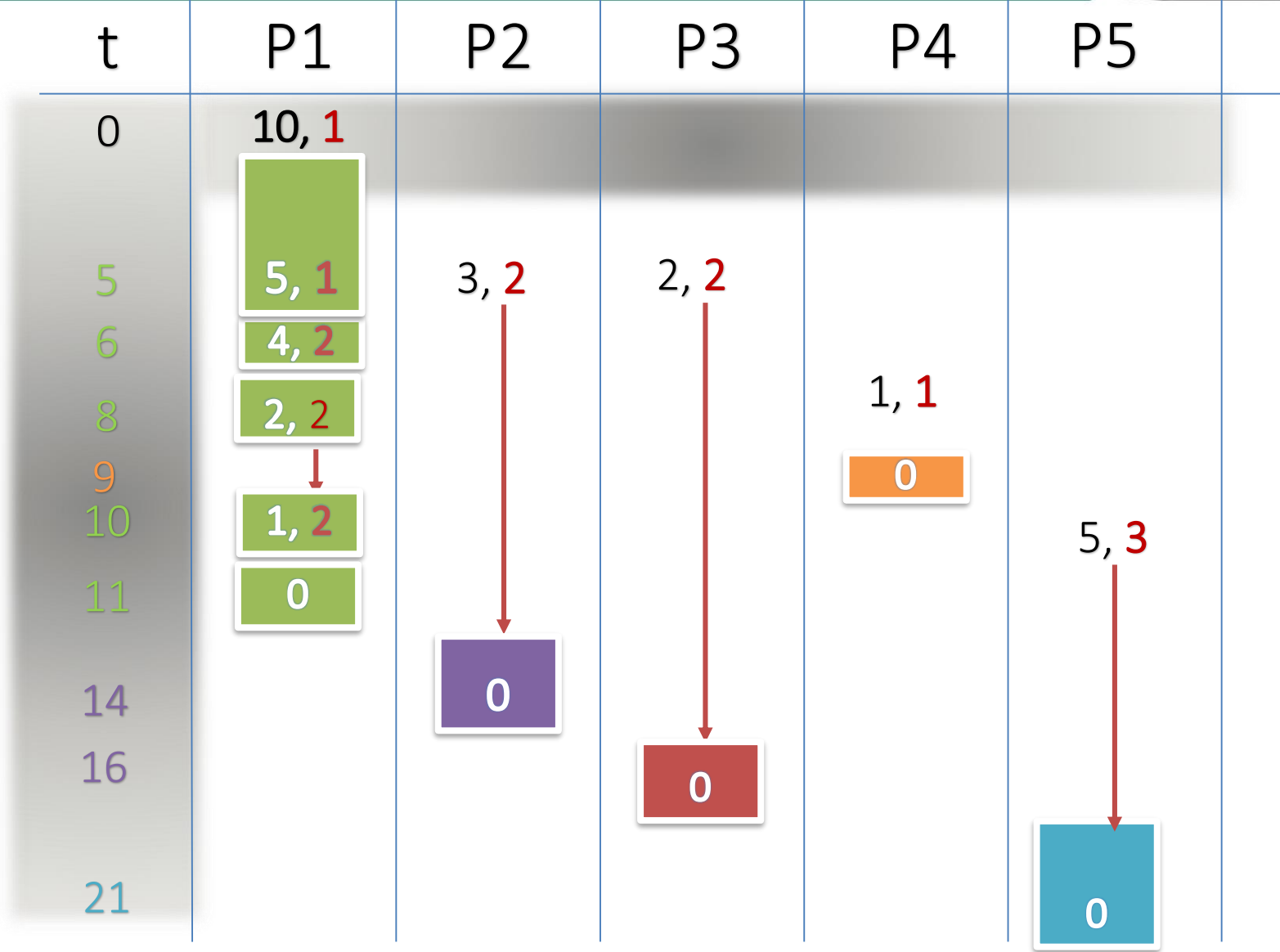


Ví dụ 2: trường hợp không độc quyền

<u>Tiến trình</u>	<u>Thời điểm đến</u>	<u>Thời gian bù đắp</u>	<u>Độ ưu tiên</u>
P_1	0.0	10	1
P_2	5.0	3	2
P_3	5.0	2	2
P_4	8.0	1	1
P_5	10.0	5	3

Priority Scheduling với SJF

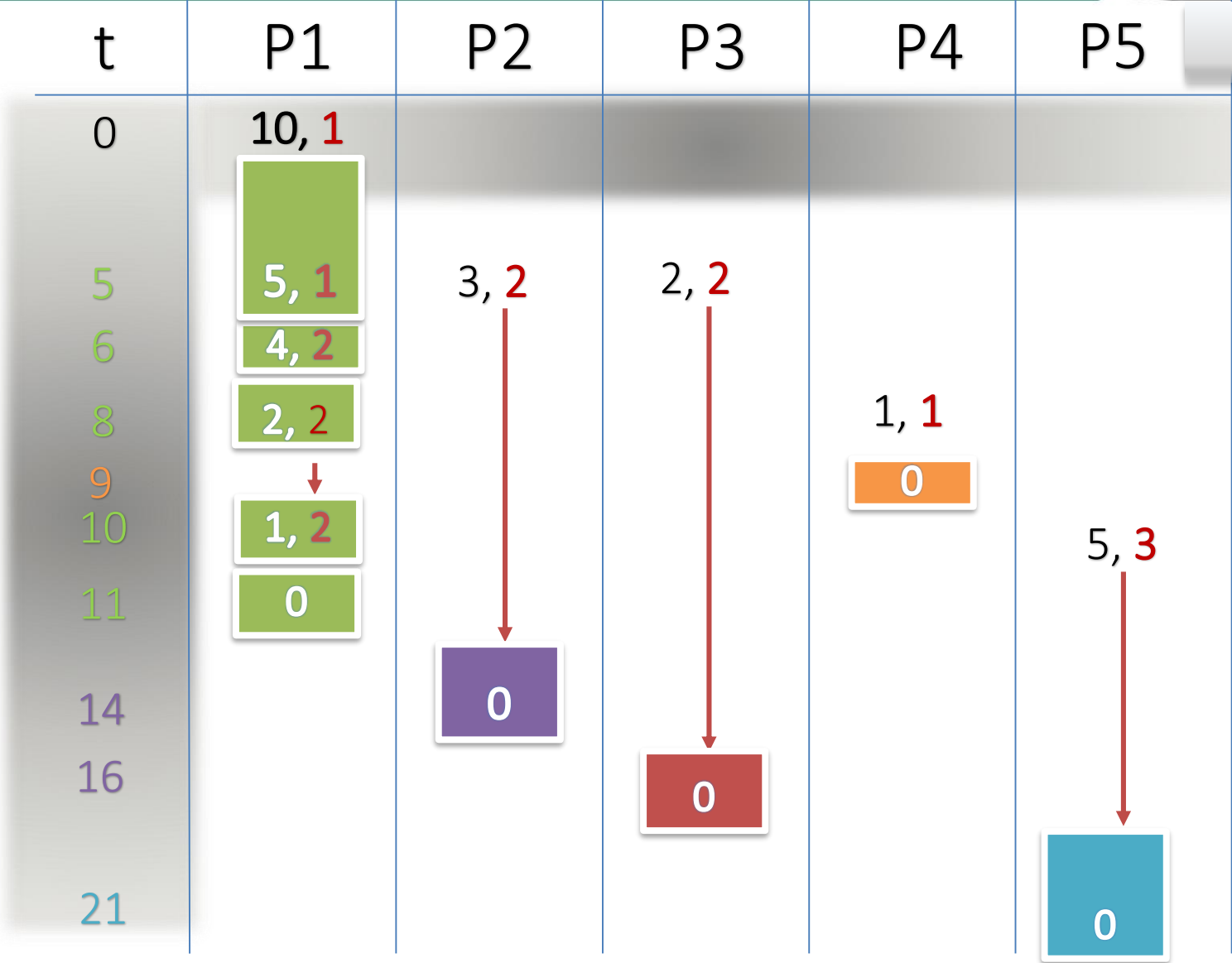
Ví dụ 2: trường hợp không độc quyền, độ ưu tiên thay đổi tối thiểu sau 6 giây và một tiến trình mới vào



Priority Scheduling với SJF



Ví dụ 2: trường hợp không độc quyền, độ ưu tiên thay đổi tối thiểu sau 6 giây và một tiến trình mới vào

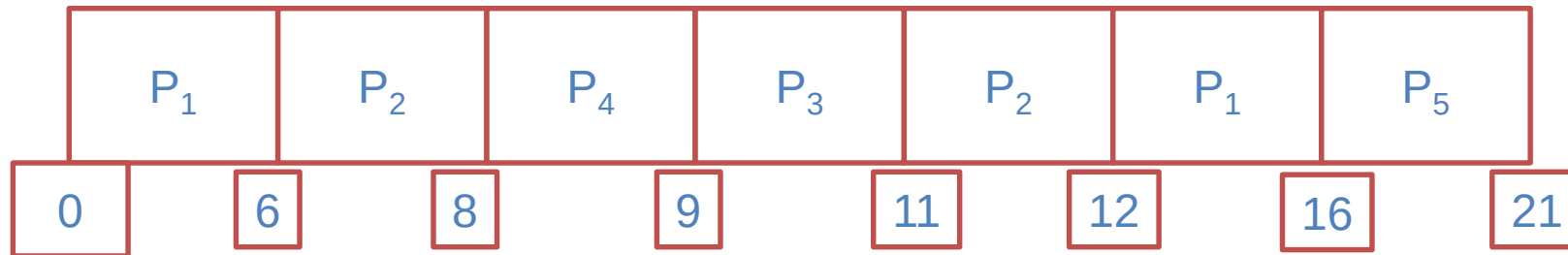


Minh họa lần lượt

Priority Scheduling với SJF



Biểu đồ Gantt là:



Thời gian chờ trung bình: $(6+4+4+0+6)/5=4$

THANK YOU

